Article

From General to Specific: A Survey of Time-Series Agent Systems

Jiaming Ma [1], Yanjun Zhao [2], Shu Wan [3,4], Hou Zhu [4], Rui Zhu [4], Ruichu Xia [5], Jiyuan Tan [5], Haofeng Ma [6], Stephen Wang [4], Jingrui He [2], and Qingsong Wen [7,8,*]

[1] University of Science and Technology of China
[2] University of Illinois Urbana-Champaign
[3] Arizona State University
[4] Abel AI Lab
[5] Stanford University
[6] University of Nottingham
[7] Squirrel AI Learning
[8] University of Oxford

* Correspondence: qingsongedu@gmail.com

Abstract

Time-series analysis supports forecasting, diagnosis, and decisions in changing environments. Practical applications require data processing, information gathering, model execution, and result validation within a coordinated workflow, motivating the development of Time-Series Agents. Building such a system for a particular scenario raises a central question: which general Agent components can be reused, which mechanisms require temporal adaptation, and which procedures must be designed around the target task? Existing surveys organize the literature by models, reasoning mechanisms, and tasks; connecting these perspectives to system construction requires an explicit account of module reuse and specialization. This survey introduces a *from-general-to-specific* taxonomy based on the scope of module reuse. Alongside LLM-side capabilities, we distinguish three Harness layers: General Harness, General Time-Series Harness, and Task-Specific Harness. These layers separate domain-independent runtime mechanisms, shared temporal capabilities, and procedures bound to task objectives. The taxonomy connects module functions, reuse conditions, and task requirements, allowing contributions from different systems to be compared and composed. We synthesize representative methods through their mechanisms, assumptions, feedback requirements, and design trade-offs, and connect these choices to development infrastructure and evaluation benchmarks. This organization guides readers from scenario requirements to module selection, system composition, and evaluation, while revealing open problems in reusable temporal mechanisms and system-level assessment. Companion Paper Cards and a structured repository make the taxonomy accessible for both human study and agent-assisted development through machine-readable module descriptions, source evidence, and official implementation links.

Keywords: Time Series; Large Language Models; Agents Systems; Harness Engineering

1. Introduction

Time-series analysis informs actions in changing environments, from demand planning to traffic control [1,2]. As new observations arrive, applications may require updated data processing, contextual evidence, analytical operations, and result validation. Recent Agent workflows connect numerical tools to iterative reasoning and evidence checks [3], and organize temporal tasks around executable

artifacts and validators [4]. LLMs coordinate these operations by interpreting requests, selecting actions, and responding to tool outputs. General Agent and Harness research develops the supporting mechanisms for tool use, memory, planning, execution, and feedback [5,6]. Temporal applications add requirements on observation order, numerical state, and information availability: workspace-based execution preserves the series used by tools [7], while contextual forecasting protocols control the evidence available for prediction [8]. Task objectives further determine the required output and evaluation criterion, ranging from forecasts to diagnostic or decision-support results [9]. The construction challenge is to determine how general interaction and execution mechanisms should acquire the temporal and task-specific capabilities required by an application.

This challenge motivates a survey organized around *from general to specific*. For a developer facing a new scenario, the key questions are: *which components can be reused, which need temporal adaptation, and which must change with the task objective?* Existing systems illustrate why these distinctions matter. Reusable execution mechanisms support task workspaces, tool calls, and artifact checks [4], while temporal workspaces and historical context preserve information needed by analytical operations [7]. Feedback is more tightly coupled to the target: forecasting-prompt refinement uses observed forecast errors [10], whereas interval-detection training evaluates anomaly agreement and false positives [11]. Adapting a forecasting workflow to anomaly analysis therefore calls for examining its execution, temporal state, and task feedback separately. A taxonomy that exposes these dependencies can turn experience from existing systems into guidance for constructing new ones.

1.1. Position relative to existing surveys

Time-series surveys supply complementary accounts of predictive architectures, foundation models, LLM adaptation, and multimodal temporal modeling [1,12–15]. Our construction question links these bodies of work: which general mechanisms remain reusable, which require temporal semantics, and which become tied to a particular task?

Recent surveys of LLM-driven time-series analysis organize the literature through complementary perspectives. One distinguishes representation, model adaptation, workflow autonomy, and analytical tasks [16]; these axes are also specified in its author-maintained taxonomy.[1] Other reviews examine reasoning topologies, objectives, and mechanisms [17], or organize Agent methods by forecasting, synthesis, anomaly diagnosis, and decision support [9]. A prediction-focused synthesis relates standalone, tool-augmented, and hybrid architectures to training, evaluation, and applications [18]. These organizing choices clarify different aspects of the field; our construction question concerns how their capabilities should be selected and combined for a particular scenario.

These surveys establish complementary views of model capabilities, reasoning, and applications. Task-oriented organization brings together methods that address similar outputs [9], while model- and mechanism-oriented organization reveals shared technical choices [16,17]. System construction additionally requires tracing how those choices depend on temporal semantics and task objectives. Our taxonomy uses the scope of module reuse to connect these perspectives: it distinguishes the runtime mechanisms that can be carried across domains, the temporal capabilities shared across tasks, and the procedures whose objectives and feedback are task-bound. This distinction provides the organizing basis for comparing and selecting components throughout the survey.

Table 1 compares explicit organizing dimensions in the closest time-series surveys. *Adaptation* distinguishes ways of changing the model or its temporal interface; *Tasks* organizes several temporal task families; *Lifecycle* connects system construction, learning or updating, evaluation, and operational use. The comparison shows complementary organizing choices: our construction view connects the

[1] Author-maintained survey taxonomy: https://github.com/CoderPowerBeyond/Agent-Prompt-TS-Survey.

model, runtime, temporal mechanisms, and output contract so that a practitioner can decide which components to retain or change.

Survey	Adaptation	Tasks	Lifecycle
From Prompts to Agents: A Comprehensive Survey of LLM-Driven Time Series Analysis [16]	✓	✓	×
A Survey of Reasoning and Agentic Systems in Time Series with Large Language Models [17]	✓	✓	×
LLM Agents for Time-Series: A Survey [9]	×	✓	×
LLM-based Agents for Forecasting and Prediction: Methods, Training, Evaluation, and Applications [18]	✓	×	✓
Ours	✓	✓	✓

Table 1. Organizing dimensions in related time-series surveys. A checkmark denotes an author-stated organizing axis or an explicit part of the review structure; a cross denotes that the dimension is not used in that role. Adaptation covers model or interface adaptation; Tasks separates multiple temporal task families beyond prediction; Lifecycle links construction, training or updating, evaluation, and operational use.

1.2. A taxonomy from general to specific

Our principal contribution is a module-level taxonomy based on the scope of reuse. **General Harness** organizes domain-independent interaction, memory, planning, tool execution, and feedback. **General Time-Series Harness Module** introduces temporal semantics and operations that can serve different time-series tasks. **Task-Specific Time-Series Harness Module** specifies procedures whose outputs, objectives, or validation rules depend on the target task. LLM-side capabilities form an independent component that interacts with these layers. Existing approaches develop both action selection through tool feedback [3] and temporal interpretation through model adaptation [19,20]; our taxonomy locates these capabilities alongside the runtime mechanisms they use. The three Harness layers describe a progression from a general runtime, through shared temporal capabilities, to a specialized task procedure.

The unit of classification is a concrete module. We examine the conditions on its inputs, state, operations, feedback, and outputs to assign it to the most specific applicable layer. A paper can contribute several modules at different layers. Synthesizing the Agent and Harness literature [5,6,17], we use five mechanism dimensions—interface and interaction; memory and context; tools and execution; planning and orchestration; and optimization and evolution—to describe module functions across these layers. Shared temporal mechanisms are examined through data management, processing, context and memory, and temporal validation. Task-specific implementations are organized around forecasting and prediction, augmentation and synthesis, anomaly detection and diagnosis, and decision support, following the task grouping in prior time-series Agent synthesis [9]. These coordinates answer complementary questions: what result is required, what mechanism produces it, and under what conditions that mechanism can be reused. Figure 4 connects the task and temporal-function views developed in the subsequent chapters.

Figure 1 maps the survey onto the construction process. Follow the arrows from the LLM side and reusable runtime to temporal contracts, task modules, evaluation, and open problems; the sequence mirrors the construction process of a Time-Series Agent. Each section answers the next design or evaluation question.

The taxonomy also structures our comparative synthesis and resource analysis. We examine representative mechanisms and their formulations to explain how methods acquire evidence, maintain state, execute tools, and use feedback, and when these choices fit a new scenario. Infrastructure and benchmarks are related to the responsibilities they support or measure. Available resources include

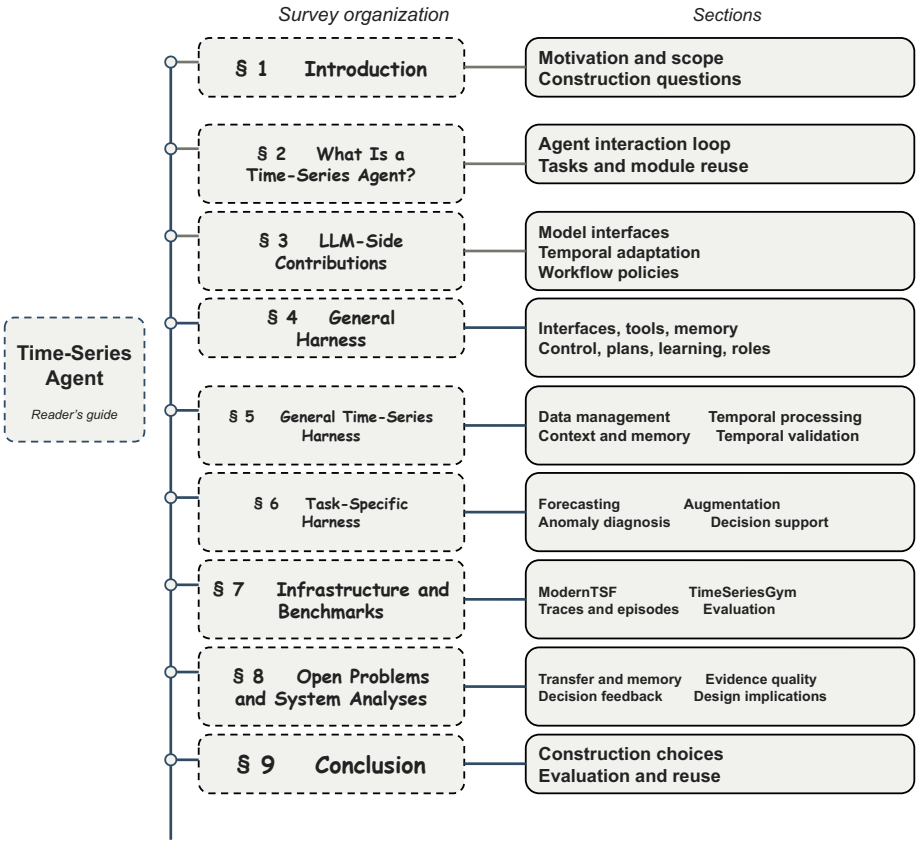


Figure 1. Organization of the survey. The sections follow the construction of a Time-Series Agent, from the LLM side and reusable runtime to temporal contracts, task modules, evaluation, and open problems. Visual layout adapted from a prior survey [21], Figure 2; the section content follows our construction framework.

numerical forecasting libraries,[2] environments that evaluate executable analytical artifacts [22], and benchmarks for evidence retrieval and contextual forecasting [8]. Connecting such resources to module and task requirements guides the path from scenario definition to component selection, system composition, and evaluation. The resulting comparisons also identify open problems in shared temporal mechanisms, experience transfer, and system-level assessment.

Figure 3 provides the landscape view of this survey. Start at the center with Time-Series Agent Systems, move outward through General Harness, General Time-Series Harness, and Task-Specific Harness modules, and then read the outer tags for representative works and resources. The radial sectors expose the mechanism and task dimensions used throughout the survey. This landscape view complements Figure 1: the former maps the literature, while the latter gives the reading order. Section 4 synthesizes the mechanisms into five runtime functions, with control, verification, and coordination discussed within planning and orchestration. The subsequent temporal and task-specific sections examine how these functions acquire domain and task requirements.

Figure 2 shows how the reviewed module contributions accumulate across the independent LLM side and the three Harness layers. Task-specific procedures account for the largest number of annotations in the coded corpus, alongside contributions to shared temporal capabilities and general execution. Each module is counted once at its assigned layer, while distinct modules from one paper may appear on different curves. This distribution motivates examining the reusable mechanisms

[2] ModernTSF repository: https://github.com/Diaugeia/ModernTSF.

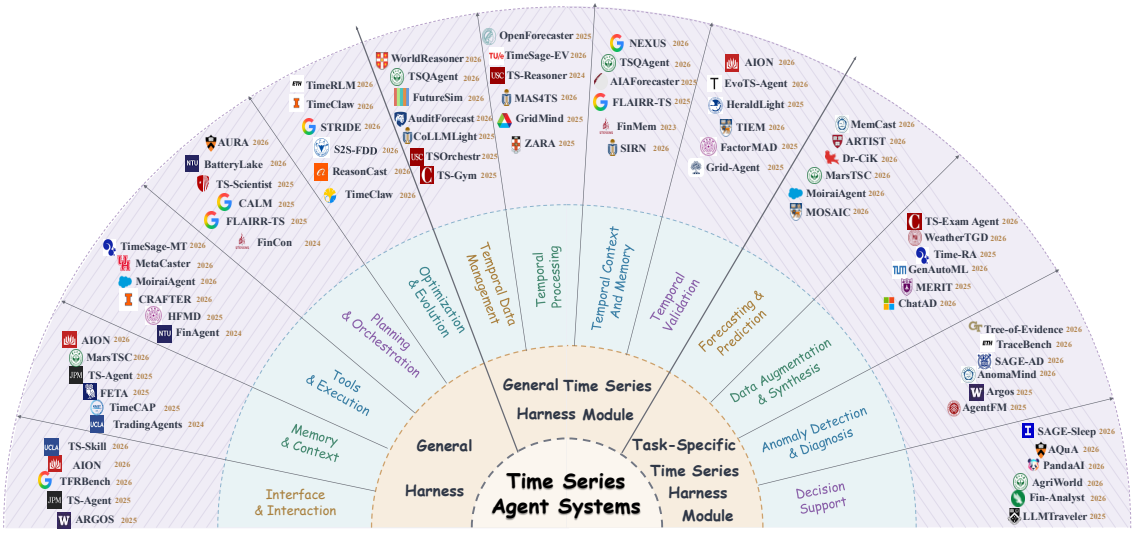


Figure 3. Landscape of Time-Series Agent Systems. The concentric layers organize the literature from the system core through General Harness, General Time-Series Harness, and Task-Specific Harness modules. Radial sectors identify the mechanism and task dimensions used in the survey, while the outer tags place representative works in these coordinates. The landscape provides a visual map of the literature; module-level assignments are recorded in the Paper Cards and discussed in the subsequent sections.

within task-oriented systems and comparing their specialization requirements. The coding procedure and complete profiles appear in Appendix A.

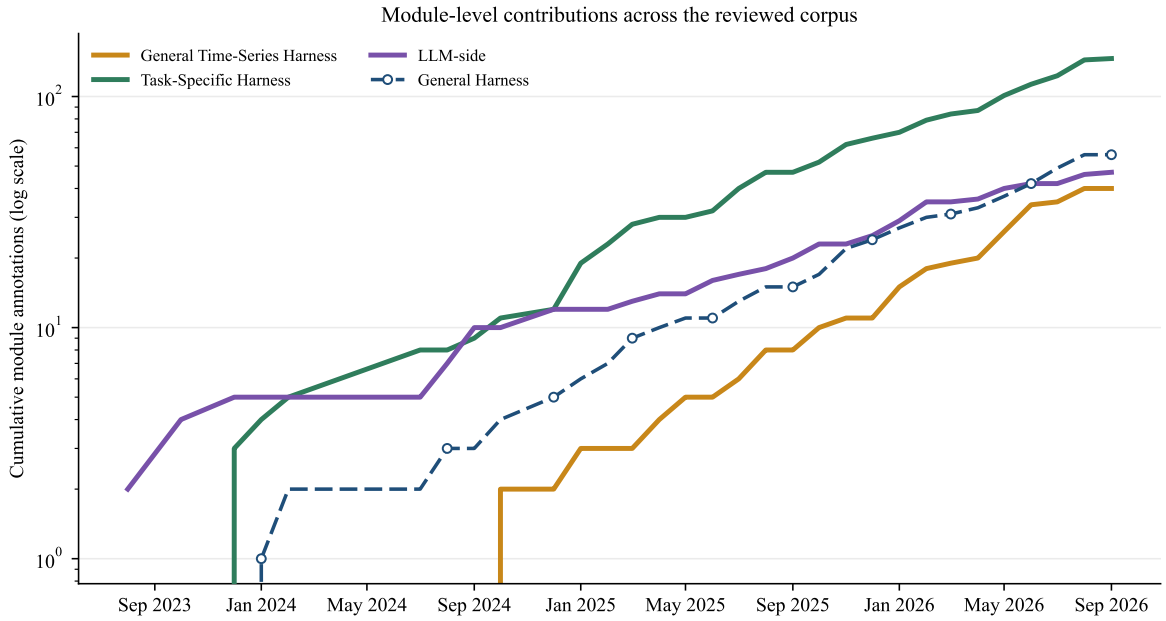


Figure 2. Cumulative source-reviewed module annotations across 134 dated records, covering methods, model training, training data, infrastructure, and evaluation. Each module counts once at its assigned layer. Dates follow the first public version, with year-only dates placed in January; the undated repository record is omitted. The logarithmic axis separates the four curves; dashed lines and hollow markers identify General Harness.

1.3. Contributions

To support inspection and continued use of the taxonomy, we organize module contributions, source passages, and official resources into Paper Cards and a structured GitHub repository.[3] The survey explains the classification and its implications for human readers; the repository provides machine-readable descriptions and resource links for retrieval and agent-assisted development. Our contributions are:

- **A module-level taxonomy from general to specific.** We distinguish three Harness layers by their scope of reuse and connect them through shared mechanism dimensions and task requirements, providing a unified account of how Time-Series Agents are composed and specialized.
- **A taxonomy-driven comparative synthesis.** We analyze representative methods through their mechanisms, assumptions, feedback conditions, and design trade-offs, explaining the boundaries of module reuse and the changes required for a target scenario.
- **Connections between classification, development, and evaluation.** We map module responsibilities and task requirements to infrastructure and benchmarks, guiding system construction and deriving open problems from cross-method comparisons.
- **Structured resources grounded in the taxonomy.** Our Paper Cards and GitHub repository link module classifications, original evidence, and official implementations for human navigation and agent-assisted development. Appendix A presents the schema, corpus statistics, and complete profiles.

2. What Is a Time-Series Agent?

A Time-Series Agent is a system in which an LLM operates through a reusable runtime and time-series-specific modules.

[3] Survey and companion resources: https://github.com/Minions-Land/Time-Series-Agents-Survey.

LLM-based Agent $\approx$ LLM + Harness,
Time-Series Agent $\approx$ LLM + General Harness + Time-Series Harness Module.

The LLM interprets the request and current observations, chooses the next tool or environment action, and uses feedback to revise its plan. The General Harness supplies interaction, memory, tools, execution, control, planning, optimization, and coordination. The Time-Series Harness Module preserves temporal state, operations, context, and validation; statistical models, time-series foundation models, databases, simulators, retrieval systems, and evaluators operate as callable components within it.

At an abstract level, the runtime can be written as

$$a_k = \pi_\theta(s_k), \qquad s_{k+1} = \mathcal{H}(s_k, a_k, o_{k+1}), \tag{1}$$

where s_k is the visible state at execution step k, a_k is a tool or environment action, o_{k+1} is the returned observation, and $\mathcal{H}$ is the Harness transition. Time-Series Harness Modules add temporal type information to each state, action, and observation.

We describe each contribution using one set of coordinates: its LLM or Harness locus, its mechanism, and its task. LLM-side contributions change how the model represents temporal inputs, produces analytical responses, or selects actions from workflow feedback. A Harness module receives one scope: General Harness for domain-independent execution, General Time-Series Harness for reusable temporal functionality, or Task-Specific Harness for a procedure tied to a task output or objective. The five mechanism dimensions in Table 3 provide a common comparison vocabulary throughout the Harness chapters. Task labels describe the outputs supported by a module. Functional labels expose temporal data management, processing, context and memory, and validation within this same record. A paper may contribute several modules, and a module may have several mechanism or task labels while contributing one unit to its assigned layer.

Task, function, and scope answer different design questions. The task specifies the required result; the function identifies the problem a module solves; and the scope records the assumptions that constrain its reuse. The four temporal functions used in Section 5 group concrete applications of these mechanism dimensions. A temporal-data interface, for example, combines interface and state mechanisms, while a temporal validator combines control, feedback, and executable checks. A module retains its scope when another task calls it; a new task-specific objective can instead motivate a distinct specialized module.

Figure 4 provides two routes through the literature. Reading a row compares a temporal need across tasks; reading a column identifies functions to combine for one task. Concrete examples help interpret these requirements: TimeSeriesScientist's Curator [23] and SAGE-AD's dual representation [24] supply shared temporal functions, while MERIT's positive-view procedure [25] and FinMem's financial memory [26] depend on their task objectives. A task column can therefore contain both GTS and TSK modules. The functions are many-to-many annotations of the same module records, and contribution counts continue to use unique module identifiers.

The lower panel illustrates how to use the grid. A demand-planning Agent first needs an execution environment, aligned historical observations, and context available at the forecast origin. It then selects or revises a forecast and passes that result to an inventory procedure with explicit costs and action constraints. ModernTSF^2 can supply numerical forecasting components, Dr-CiK [8] can test research with external context, and TimeSeriesGym [22] can test the production of executable analytical artifacts. Inventory outcomes require a scenario environment with the relevant demand, lead-time, cost, and stock constraints. Section 7 develops this matching of requirements to resources.

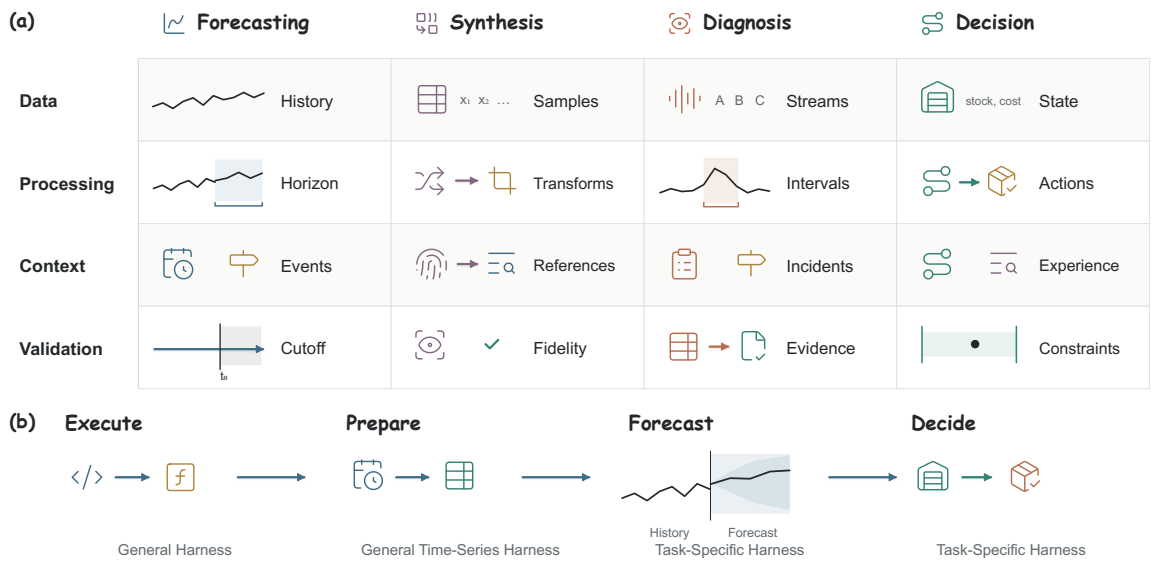


Figure 4. From task requirements to Harness modules. (a) Columns identify task families and rows identify shared temporal functions. Read a column to assemble requirements for one task, or a row to compare how a function serves different tasks. Shaded regions and solid boundaries mark a forecast horizon, a diagnostic interval, information beyond the forecast origin t_0, and an admissible action range. (b) A demand-planning workflow combines general execution, shared temporal preparation, forecasting, and inventory decisions. The forecast panel separates observed history from a schematic prediction and its range. The grid synthesizes requirements illustrated by TimeSeriesScientist [23], SAGE-AD [24], MERIT [25], and FinMem [26].

1. **Specify the scenario.** State the data sources, event and availability times, channels, units, missingness policy, and the decision or analysis that follows the output.
2. **Write the task contract.** Define the output, horizon, admissible information, constraints, evaluation metric, and review authority.
3. **Select the runtime.** Choose General Harness mechanisms for interaction, memory, tools, execution, control, planning, optimization, and coordination.
4. **Add temporal semantics.** Introduce temporal state, operations, provenance, chronology guards, and validators wherever a generic runtime leaves the meaning ambiguous.
5. **Instantiate and evaluate.** Add task-specific modules, then evaluate process validity, temporal validity, task performance, resource use, failures, revision, and downstream decision utility.

This sequence is the practical meaning of the survey's hierarchy. The same runtime can support financial forecasting, traffic prediction, industrial diagnosis, or demand planning; the temporal and task contracts determine which modules and benchmarks must change.

Table 2 defines the notation shared by the method descriptions. Temporal indices describe the data and information boundary, while execution indices describe the Agent loop; method-specific training variables are defined where they are introduced.

Symbol	Meaning	Layer	Use
$x_{1:T}$	Multivariate time series with T observations, channels, units, and timestamps.	LLM / GTS	Input to representation, temporal operations, and task modules.
t, τ, H	Event time, decision or forecast-origin time, and forecast horizon.	GTS / TSK	Define chronological availability and task output intervals.
$\mathcal{I}_\tau$	Information admissible at time τ, including availability and provenance.	GTS	Governs retrieval, memory, validation, and leakage control.
s_k, a_k, o_{k+1}	Agent state, action or tool call, and returned observation at step k.	GH	Describe the reusable state–action loop.
$\mathcal{V}$	Validator or verifier that returns a pass, fail, finding, or escalation decision.	GH / GTS / TSK	Checks process, temporal, and task-specific obligations.
$(\widehat{y}, \widehat{z}, \widetilde{x}, a^\star)$	Forecast, anomaly decision, generated series, and selected action.	TSK	The four representative task outputs used in Section 6.

Table 2. Unified notation for the module-level method descriptions. GH denotes General Harness, GTS denotes General Time-Series Harness, and TSK denotes Task-Specific Harness.

Sections 3–7 develop these choices from model interfaces and reusable runtime mechanisms to temporal modules, task procedures, and evaluation resources.

3. LLM-Side Contributions for Time-Series Agents

Within the Agent loop, the LLM interprets observations, selects actions, and responds to tool or environment feedback. Temporal model adaptation provides one way to improve particular parts of this interaction. We review three intervention points: the representation presented to the model, the temporal instructions used to train it, and the policy for operating an analytical workflow. The design question is which capability the application needs from the model and which interface connects it to the runtime. A learned encoder needs matching numerical preprocessing; a trained workflow policy needs the tools, observations, and feedback used to define its actions.

Figure 5 locates these interventions within the interaction loop. The upper row distinguishes input alignment, temporal supervision, and workflow learning; the lower loop shows how the resulting model still relies on Harness execution, state, and feedback. The four evaluation targets at the bottom connect each learned capability to the evidence needed to assess it.

3.1. Temporal representation and alignment

Time-LLM [19] makes a frozen language backbone operate on temporal patches. It constructs a compact set of text prototypes E' from the pretrained word-embedding space and uses the temporal patch embeddings as attention queries. For channel i and attention head k, its reprogramming operation is

$$Q_k^{(i)} = \widehat{X}_P^{(i)} W_k^Q, \quad K_k^{(i)} = E' W_k^K, \quad V_k^{(i)} = E' W_k^V, \qquad Z_k^{(i)} = \mathrm{softmax}\left(\frac{Q_k^{(i)} K_k^{(i)\top}}{\sqrt{d_k}} \right) V_k^{(i)}. \tag{2}$$

Here d_k is the key dimension of attention head k. The text prototypes provide combinations of language-space cues for local temporal patterns. After aggregation and projection, the reprogrammed patches enter the frozen backbone alongside Prompt-as-Prefix context describing the task, domain, and input statistics. An output projection produces the forecast and is trained with the input transfor-

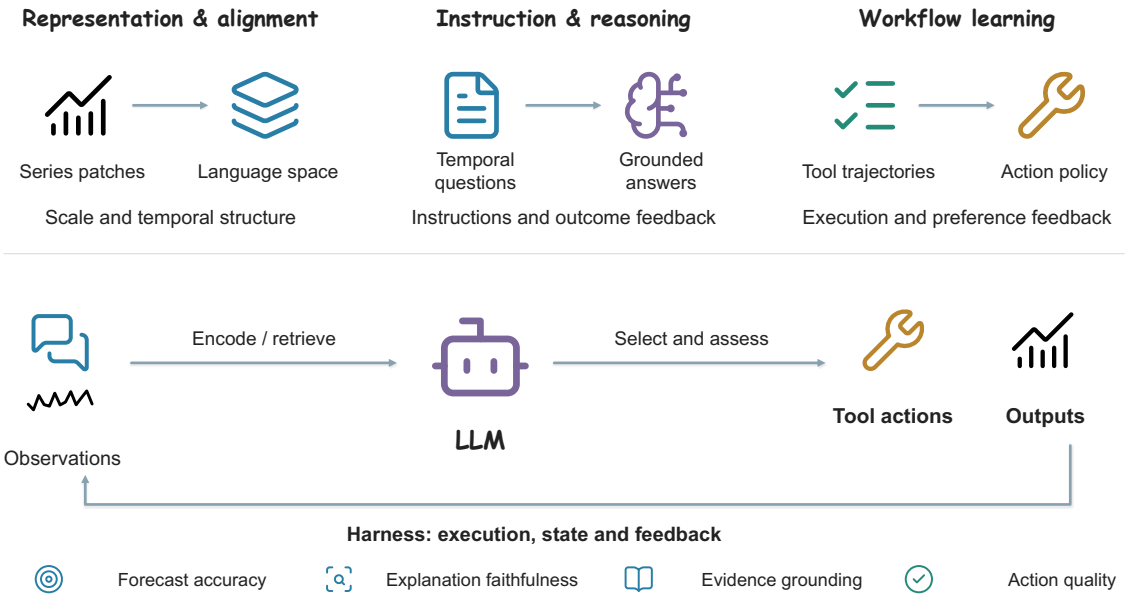


Figure 5. LLM-side intervention points in a Time-Series Agent. Representation learning connects numerical inputs to language space; temporal supervision shapes analytical responses; workflow learning shapes action selection. These routes can overlap within one method, as illustrated by temporal interfaces and supervision in ChatTS [20]. The lower loop separates model contributions from the Harness that executes calls and returns observations. Forecast accuracy, explanation faithfulness, evidence grounding, and action quality require distinct evaluation evidence.

mations under prediction loss. LLM4TS [27] likewise studies adapting a pretrained language model to temporal inputs, illustrating how the learned numerical interface can be developed before the Agent workflow is assembled.

ChatTS [20] develops a conversational temporal interface. Fixed-size patches pass through a five-layer MLP into the text-embedding space and are inserted among the text tokens at their corresponding series positions. Its value-preserving normalization couples normalized patches with scale and offset information in the prompt. For a nonconstant series, the transformation and its reconstruction are

$$x_t^{\text{norm}} = \frac{x_t - x_{\min}}{x_{\max} - x_{\min}}, \qquad x_t = (x_{\max} - x_{\min})x_t^{\text{norm}} + x_{\min}. \tag{3}$$

This mathematical restatement of the paper's normalization makes the role of the prompt metadata explicit: a statement about magnitude can refer to the original scale while the encoder processes normalized values. In an Agent, the input layer must preserve this correspondence between a numerical series and its language-facing description.

Agentic Retrieval-Augmented Generation for Time Series Analysis [28] conditions specialized language models through differentiable prompt pools. Retrieved continuous prompt embeddings are concatenated with numerical inputs and passed through a learnable projection. The pattern store thus participates in model adaptation: its learned representation determines how historical temporal patterns condition the predictor. This choice suits model-facing temporal conditioning, whereas external episode retrieval preserves inspectable records for the running Agent.

For numerical forecasting, Time-LLM [19] offers prototype-based reprogramming of a frozen backbone, whereas ChatTS [20] learns a direct patch-to-language interface for conversational analysis. Equation 2 identifies where numerical patches interact with language prototypes; Equation 3 identifies the scale information that must accompany normalized inputs. The choice therefore depends on the required output: a numerical forecast head or answers to temporal questions. The Harness

supplies window construction, channel and unit metadata, consistent preprocessing, and routing between the learned interface and downstream tools.

3.2. Temporal instruction and reasoning

A temporal encoder becomes useful for analysis through supervision that links signal structure to questions and explanations. ChatTS [20] generates synthetic series with selected temporal attributes, constructs matching text, and evolves instructions for understanding and reasoning. Controlling the generated attributes supplies known temporal properties around which analytical instructions can be built.

Towards Time-Series Reasoning with LLMs [29] follows a related learning route through temporal encoding and reasoning supervision. Its encoder warm-up and subsequent language-model adaptation connect temporal perception to explicit reasoning traces. Time-R1 [30] specializes this idea for forecasting: supervised warm-up teaches structured reasoning and complete forecast outputs, followed by reinforcement fine-tuning with format, length, and numerical feedback. Its MSE-based reward maps normalized forecast error to a bounded signal,

$$\gamma_{\mathrm{MSE}} = 2\left(1 - \frac{1}{1 + e^{-2\,\mathrm{MSE}}}\right), \tag{4}$$

which equals one at zero error and approaches zero as error grows. This bounded transformation compresses differences among high-error forecasts; the normalization of predictions and targets consequently determines the effective sensitivity of the reward. Equation 4 illustrates an outcome-based training choice: forecast error supplies a continuous signal alongside format, output-length, and temporal-structure rewards. The accompanying GRIP optimizer uses nonuniform sampling and adaptive weighting over reasoning paths. Time-R1 [30] evaluates forecasting with MSE and MAE and ablates its reward components (Tables 2, 4, and 6); these experiments support comparisons of numerical performance and training choices. Explanation faithfulness requires an additional test of whether the stated reasoning reflects the evidence and computation that produced the forecast.

STRIDE [31] connects language reasoning to a separate numerical foundation model. A teacher constructs reference reasoning from training windows and their pseudo-future targets; a smaller language model learns this reasoning. Its last hidden states H are mean-pooled and projected into a numerical-model-compatible reasoning vector e_R, which is fused with native time-series embeddings E_{TS}:

$$h_R = \frac{1}{L}\sum_{i=1}^{L} H_i, \qquad e_R = h_R W_{\mathrm{proj}}, \qquad E_{\mathrm{fused}} = e_R \oplus E_{\mathrm{TS}}, \qquad \mathcal{L}_{\mathrm{total}} = \alpha\mathcal{L}_{\mathrm{CE}} + \beta\mathcal{L}_{\mathrm{Quantile}}. \tag{5}$$

The fusion operation uses sequence prefixing or initial-state substitution according to the forecaster architecture. The joint objective updates the numerical model and projection together with LoRA parameters in the language model. This design lets a language-derived reasoning representation influence continuous forecasts while retaining a dedicated numerical decoder.

The practical choice is between supervising temporal descriptions, rewarding completed forecasts, and coupling language representations to a numerical decoder. ChatTS [20] supplies attribute-based supervision for temporal questions; Time-R1 [30] ties learning to forecast outcomes; STRIDE [31] retains a dedicated forecaster while injecting a learned reasoning representation. In Equation 5, the language and quantile losses make this joint adaptation explicit. Their use requires different Harness support: preservation of series–text correspondence, parsing and chronological scoring of forecasts, or synchronization of language features with the numerical model's inputs.

Evaluation follows the capability being selected. Forecast accuracy concerns predicted values; explanation faithfulness concerns whether the explanation reflects the model's actual computation; ev-

idence grounding concerns whether its claims are supported by the supplied series or documents; and action-selection quality concerns feasible, useful tool or environment actions. These distinctions motivate complementary checks: held-out numerical errors, controlled interventions on claimed reasoning factors, verification of cited observations and their availability times, and execution-based action evaluation. Section 7 connects these requirements to evaluation resources.

3.3. Learning to operate temporal workflows

Workflow training changes the model that selects or assesses actions. TSOrchestra, introduced in *Conversational Time Series Foundation Models* [32], trains a language-model judge for iterative ensemble optimization. A numerical optimizer proposes nonnegative weights over forecasting models that sum to one. The judge inspects metrics, weights, and optimization history to decide whether to continue or accept a configuration. Supervised fine-tuning uses real trajectories and augmented boundary cases; subsequent GRPO training incorporates held-out forecasting performance and explanation feedback. The trained judge controls numerical optimization by deciding when an ensemble configuration is ready to accept.

CastFSR [33] trains a broader policy over the Fast–Slow–Reflect workflow. Supervised distillation teaches executable stage sequences, followed by multi-turn reinforcement learning over complete trajectories. Training checks tool calls, stage order, timestamps, and completed forecasts before applying forecasting feedback. At inference, the policy still interacts with the numerical forecasting and contextual-retrieval tools described in Section 6. TimeART [34] similarly combines a trained reasoner with an analytical toolkit, connecting policy learning to explicit temporal operations.

ChatAD [35], Section 4.2, develops an anomaly-reasoning model through LoRA-based supervised training and TKTO preference optimization. Feedback combines format compliance, task correctness, fluency, and human preference into score S. A smooth preference weight mixes desirable and undesirable response losses:

$$\alpha = \sigma(\kappa(S - \eta)), \qquad \mathcal{L}_{\mathrm{TKTO}} = \mathbb{E}_{(x,y,d)\sim\mathcal{D}}[\alpha\mathcal{L}_D(\pi_\theta, x, y) + (1-\alpha)\mathcal{L}_U(\pi_\theta, x, y)]. \tag{6}$$

At $S = \eta$, the two losses receive equal weight; increasing κ makes the transition between preferred and rejected responses sharper. ChatAD [35] uses this soft assignment to address noise near a hard preference threshold (Section 4.2.2, Eqs. 3–5). The composite score combines LLM assessments of format, correctness, and fluency with human preference. Thus Equation 6 compares graded preference supervision with a binary accept/reject decision: its practical requirements include a consistent scoring rubric and calibration of the score relative to η. ChatAD's evaluation separates detection accuracy, F1, and false-positive rate from open-ended reasoning scores and format-parsing rates (Section 4.3.2, Tables 3–4). The latter measure judged answer quality and parseability; checking explanation faithfulness additionally requires evidence about how the answer was produced. The synthetic-dialogue agents create supervision, while the resulting ChatAD model supplies the learned anomaly-reasoning capability. AnomaMind [11] also combines a trained detector with an external evidence-acquisition workflow; its task-specific feedback and detection equations are developed in the anomaly section.

These methods differ in the learned role and the feedback available to train it. TSOrchestra [32] is useful when a judge must accept or continue numerical ensemble optimization; CastFSR [33] learns decisions across a prescribed tool workflow; ChatAD [35] adapts generated analytical responses using graded preferences. A developer therefore selects both a model and its required feedback interface. The Harness must expose optimizer histories or tool observations, retain intermediate state, enforce valid calls and temporal boundaries, and record outcomes for evaluation. For a workflow policy, action quality is tested through executed trajectories and their task outcomes, while the accompanying explanation is assessed separately.

4. General Harness

The General Harness connects the LLM's action choices to executable operations, persistent state, and feedback. We synthesize five recurring runtime functions from the Agent and Harness literature [5,6,9,17,18,36]. Table 3 associates each function with a design question and the operations it includes. The dimensions can coexist: an analytical episode may retrieve context, plan a sequence, execute tools, and revise an artifact using validation feedback. The examples below examine how time-series systems implement these functions and which parts of their implementation can support another task. Temporal requirements and task objectives subsequently determine how these runtime choices are specialized in Sections 5 and 6.

Table 3. Five General Harness functions and their associated design questions. A system may combine several dimensions within one workflow.

Dimension	Question	Elements
Interface and Interaction	How does the agent receive state and act on its environment?	Reasoning interfaces, action interfaces, environment modeling, user interaction, and state–action loops.
Memory and Context	How does the agent preserve and organize information across steps?	Context management, working and long-term memory, state stores, retrieval, and experience reuse.
Tools and Execution	How does the agent invoke external capabilities and execute actions?	Tool registries, APIs, retrieval, code execution, databases, forecasting models, and simulators.
Planning and Orchestration	How does the harness plan, organize, and coordinate actions to achieve a goal?	Task decomposition, planning, search, workflow orchestration, execution control, delegation, and multi-agent coordination.
Optimization and Evolution	How does the agent or Harness improve from experience?	Prompt, tool, memory, policy, and Harness optimization; reflection and trajectory-driven evolution.

Figure 6 follows an interaction episode around the LLM: the interface supplies observations, planning organizes actions, tools execute them, and memory retains evidence. The lower band shows how recorded trajectories can improve prompts, memory, or tools across episodes. The five functions in Table 3 therefore describe both running a workflow and improving its reusable components.

The five functions organize the evidence below. Each subsection first states the general mechanism and then points to its use in a Time-Series Agent. A work may instantiate any subset of these dimensions.

4.1. Interface and Interaction

The interaction interface defines what the Agent observes, which actions it can request, and how the result returns to the next reasoning step. TS-Agent [3] uses an iterative thought–action–observation loop: the reasoner chooses its next step after receiving the structured result of the preceding analytical operation. This supports questions whose required computation becomes clear during analysis. AION [4] starts from a more explicit interface, consisting of a task file, accessible workspace, and validator. The task file declares the required artifact and constraints before work begins.

These designs support different user interactions. An exploratory request benefits from successive tool observations and follow-up questions; a submission-oriented task benefits from an explicit output specification and acceptance check. Both require a stable action interface so that the LLM can identify an operation, supply its arguments, and interpret success or failure. The numerical operation

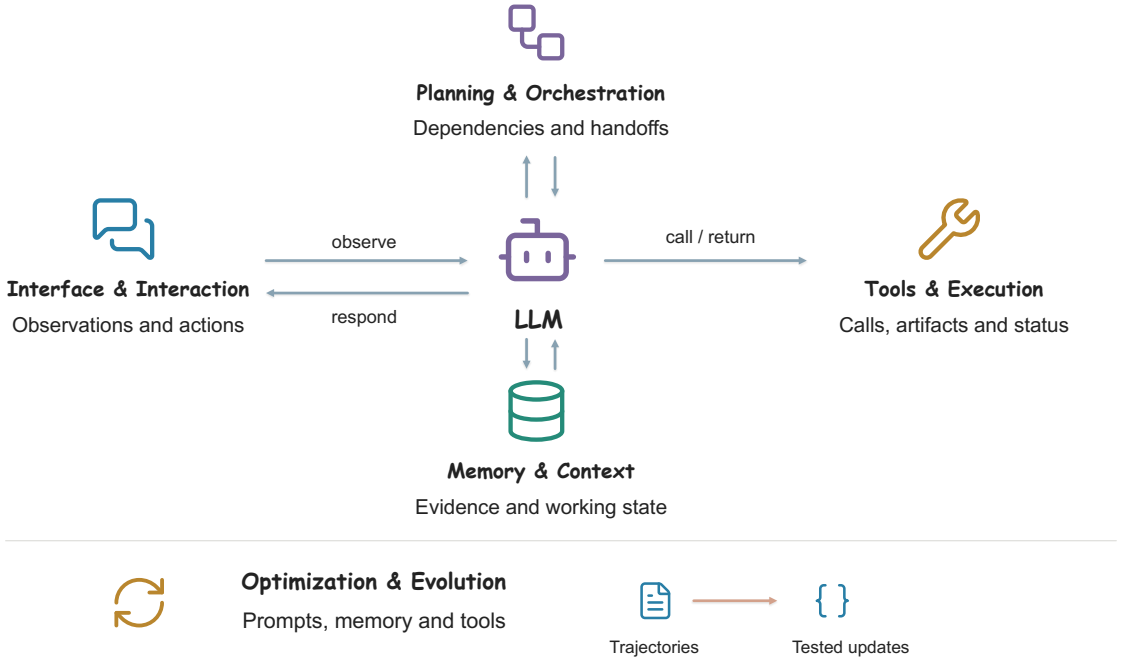


Figure 6. Five functions of the General Harness, synthesized from Agent and Harness research [5,6]. The central LLM interacts with planning, tools, memory, and the observation interface during execution. Optimization and evolution use recorded trajectories to update reusable assets across episodes. Planning and orchestration includes execution control, verification checkpoints, and coordination between roles.

and its temporal arguments are developed in the following section; the interaction loop can remain the same as the analytical objective changes.

4.2. Memory and Context

Memory determines which earlier observations remain available to later steps. TS-Agent [3] retains an evidence log so that subsequent reasoning can use completed analytical results. AION [4] records progress, decisions, and structural state around execution and review. Such records serve distinct purposes: the evidence log supports an answer, while the progress record supports continuation after interruption or handoff.

TradingAgents [37] addresses memory through structured reports. Each role extracts the information it needs and returns a completed report to the shared workflow. This gives later roles a stable summary with an identifiable author and purpose. Compared with an unrestricted conversation history, the report boundary makes the input to each decision easier to inspect. In a new Agent, the design choice is which information should remain as raw tool output, which should become a compact report, and which should persist across episodes. Signal-aware retrieval adds a further temporal requirement and is discussed in Section 5.

4.3. Tools and Execution

Execution converts a selected action into a computation and returns its result, artifacts, and status. TimeSage's reference runtime in TimeSage-MT [38] separates the LLM Planner and Reporter from deterministic validation, profiling, execution, and evaluation. The planner selects a workflow, while the executor carries out its operations. This division makes it possible to inspect whether an error arose in the chosen workflow or in the implementation of an operation.

MetaCaster's Forecaster Trainer [39] demonstrates execution at the level of computational jobs. It assembles jobs indexed by model, hyperparameters, and dataset, queues them, assigns available GPUs,

monitors processes, and recovers errors or interrupted runs. The reusable mechanism is scheduling and recovery; the job's forecasting objective is supplied by the task procedure. For an engineering Agent, this distinction separates the ability to complete a computation from the quality of the model produced by that computation.

4.4. Planning and Orchestration

Planning specifies dependencies among the operations needed to deliver an artifact. TS-Agent [3] chooses actions incrementally from accumulated observations. TS-Reasoner [40] constructs an ordered executable plan and revises it after failure. The first approach makes the next action sensitive to newly observed evidence; the second provides an explicit plan that can be inspected and rerun as a whole.

AION [4] extends planning to the lifecycle of work through protocols for execution, review, rechecking, memory updates, and stopping. The resulting design separates exploratory choices from required checkpoints. For example, an Agent may explore several analytical routes while all routes must still pass an artifact validator. This is useful when the practitioner needs both flexibility in solving the task and a repeatable acceptance process.

Control determines whether the Agent continues, revises its work, requests review, or terminates. TS-Reasoner [40] provides an executable example: its task decomposer produces an ordered operator plan, the executor runs it, and an execution error triggers a new end-to-end plan. The error message is actionable feedback about the current procedure. AION [4] adds artifact-level validity checks and a separate scored evaluation, allowing a candidate to be returned for revision before acceptance.

The source and timing of feedback determine what can change during an episode. An execution error is immediately available; a resolved forecast error is available only after the target window has been observed. FLAIRR-TS [10] uses recent ground truth to refine forecasting prompts and is therefore developed with the forecasting methods. A system that combines these controls should expose both the retry policy for failed operations and the schedule under which task outcomes become available for learning.

Coordination defines responsibilities and the information passed between roles. TradingAgents [37] uses structured report-based communication so that each role consumes a defined state and returns a completed result. AION [4] makes work, review, rechecking, and stop signals explicit in the communication protocol. These arrangements connect the ownership of an artifact to the condition under which the next role can act.

The task determines the content of the handoff. In FinCon [41], analysts and a manager aggregate financial evidence before portfolio actions; in TimeSeriesScientist [23], curation and planning outputs guide forecasting and reporting. Their financial or forecasting objectives are analyzed in Section 6. These examples suggest defining a role around the artifact it must deliver and the check that permits the next role to proceed. This makes handoffs inspectable and gives failures a recovery point. Across these runtime functions, the shared design objective is to keep action selection, execution state, and feedback connected.

4.5. Optimization and Evolution

A Harness can improve through changes to external experience, prompts, or executable tools. *TimeClaw: A Time-Series AI Agent with Exploratory Execution Learning* [7] compares alternative executed trajectories and distills useful experience into a state available to later episodes. TIMECLAW [42] develops a tool-promotion mechanism: a code-specialized LLM receives clustered trajectories and task descriptions, extracts a recurring procedure, and produces code with an invocation description.

The candidate is tested on held-out similar tasks and enters the registry after meeting the success threshold.

These mechanisms change different reusable assets. Experience distillation changes the guidance available to the reasoner, while tool promotion changes what the executor can call. Both require a record of the source episodes and the criterion used to accept an update. The procedure's analytical content determines its temporal or task scope; the mechanism for extracting, testing, and installing the update remains reusable. The focused TimeClaw analysis gives the source equations for the experience update.

5. General Time-Series Harness

The General Time-Series Harness specializes the General Harness functions through temporal order, index, frequency, window, horizon, regime, availability time, and leakage control. We organize these requirements around the temporal-grounding mechanisms of AION [4] and the runtime-native operations of TIMECLAW [42]. These temporal mechanisms build on the interaction, execution, and feedback interfaces of the General Harness.

The contract boundary makes the distinction explicit. General Harness specifies how an Agent stores state, invokes tools, coordinates roles, and verifies outputs. General Time-Series Harness specifies what a valid temporal state or operation means: which observations were available, how a window is aligned, which frequency and units apply, and how a revision is represented. TIMECLAW [42] and AION [4] make this boundary explicit through time-series-native execution, temporal grounding, and reliability mechanisms.

For a temporal state at decision time τ, the admissible information can be represented as

$$\mathcal{I}_\tau = \{(x_i, t_i, v_i) : v_i \leq \tau\}, \tag{7}$$

where t_i is the event time and v_i is the time at which the value became available. The availability boundary admits information published by τ, including a known future calendar event; the task specifies which observation and target windows may be used.

Figure 7 connects the four shared functions through the objects they manage. Read the upper timeline first: availability time determines whether a source can be used at decision time τ. The middle row follows timestamped values, a selected processing window, an aligned event, and an evidence-admission check. The lower paths connect the LLM to requests and returned observations. These operations implement the temporal grounding and evidence checks emphasized by AION [4] and TIMECLAW [42].

The four functions below organize the temporal requirements encountered when constructing an Agent: managing its data, operating on the series, retrieving context, and validating temporal evidence. They are a reading view over the five functions in Table 3. Data management draws on interfaces and state; processing on tools and workflow; context on memory and retrieval; and validation on control and feedback. A workflow can combine several functions, and coordination or optimization can act on any of them. Section 6 examines how target outputs specialize these shared capabilities.

5.1. Temporal Data Management

Before an Agent can compare signals or call a model, its inputs must identify what was measured, when it was measured, and how the stored values should be interpreted. The input schema therefore includes timestamps, sampling intervals, units, missingness, and revision information. These properties form the interface between the raw source and every later operation. AION [4], Section 3.3.1,

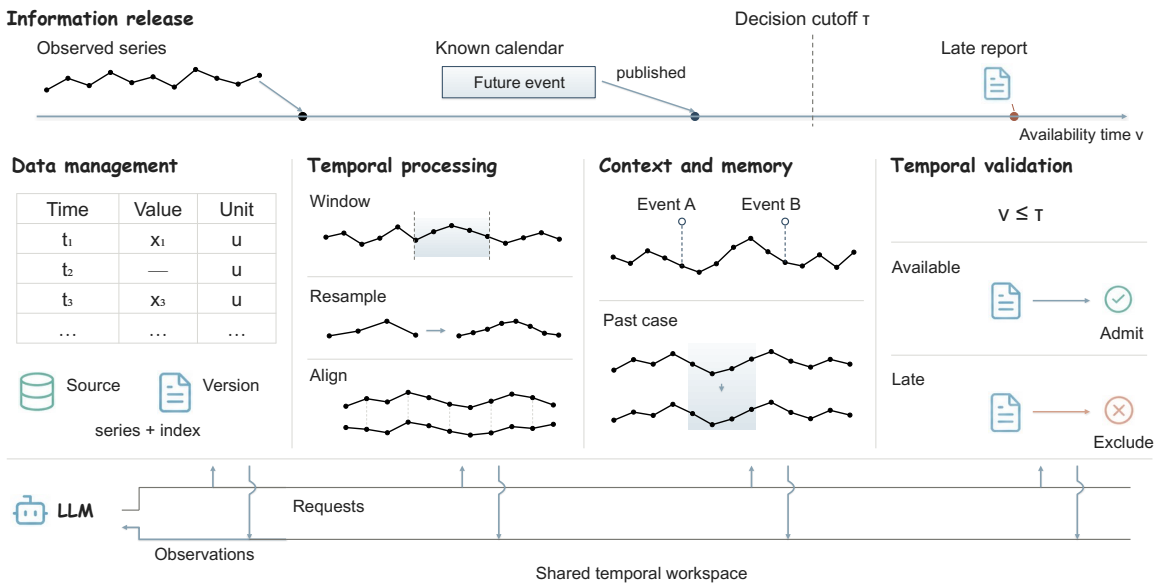


Figure 7. A temporal workspace connecting sources, operations, and the LLM. The upper timeline separates available evidence from a late release at decision time τ. The middle row illustrates timestamped data and versions; windowing, resampling, and alignment; event context and retrieved cases; and evidence-admission checks. Requests and observations connect these shared functions to the LLM. This synthesis follows the temporal-runtime requirements of AION [4] and TIMECLAW [42].

places this interface before model selection: the task must resolve the observation unit, visible history, cutoff, requested horizon, granularity, and output frequency. The resulting requirements determine which data are admissible and which operations can satisfy the request.

BatteryLake [43] makes measurement semantics explicit through a canonical sample schema. Time, voltage, current, temperature, capacity, and step type have fixed units and domains; the current-sign convention distinguishes charging from discharging. This supports consistent interpretation of heterogeneous battery files before downstream health or lifetime analysis. LEMAD [44] addresses a related ingestion problem for operational services: its preprocessing module aligns timestamps, normalizes formats, and handles missing values across heterogeneous streams. The common design choice is to resolve measurement and alignment conventions at ingestion, where subsequent tools can inherit them.

Stored data also need a representation that preserves numerical precision. TIMECLAW [42], Section 3.2, keeps the full time-series object in a task-local workspace. Tools run on that object and return compact structured observations to the LLM. SAGE-AD [24], Section 3.2, likewise separates the original series used by numerical tools from a compressed summary used in prompting. These mechanisms let the language interface remain compact while the analytical operations retain access to the measurements. The anomaly-specific use of SAGE-AD's resulting evidence is discussed in Section 6.3.

Streaming inputs introduce an additional state requirement. CALM [45] buffers incoming observations per key before its stateful sliding-window function emits the ordered window. Without this step, out-of-order arrival changes which observations enter a computation. This mechanism complements a canonical schema: the schema defines the meaning of each sample, while the buffer preserves membership and order as samples arrive. For a new application, the ingestion design follows the source of ambiguity: measurement schemas address heterogeneous files, numerical workspaces preserve tool inputs, and ordered buffers handle streaming arrivals. These choices can be combined as the data sources require. Testing should vary arrival order, units, missing entries, or timestamp formats and inspect the numerical object delivered to the next operation.

5.2. Temporal Processing

Processing converts a stored series into the evidence needed for analysis. Common operations include window selection, resampling, decomposition, smoothing, correlation analysis, and feature extraction. Their inputs and outputs must preserve the relation to the source sequence: the same numerical array may describe levels, differences, normalized values, or an aggregate over a longer interval. A useful tool interface therefore exposes both the operation and the temporal meaning of its result.

TS-Agent [3] supplies analytical APIs that operate directly on native numerical series and return structured findings. TS-Reasoner [40] wraps statistical tools, learned models, and numerical utilities in a consistent calling structure. TimeART [34] uses atomic tools for computation-intensive temporal operations. These designs differ in their inventories and calling policies, but each makes a numerical operation independently accessible to the Agent. A developer can compose these operations around the target workflow, inspect their intermediate outputs, and reuse them across analytical requests.

TimeSeriesScientist's Curator [23], Section 3.2, illustrates how inspection determines the operation. Given a series D, it produces data statistics S, missing-value information M, outlier findings O, and a treatment strategy π:

$$Q = A_f(D) = (S, M, O, \pi), \qquad \widetilde{D} = \phi_\pi(D). \tag{8}$$

The processed series then supports visualization, decomposition, and autocorrelation analysis. These diagnostics precede the forecasting-specific model selection in Section 6.1. For example, identifying missing intervals changes the preprocessing plan, whereas ranking candidate forecasters requires a target horizon and validation loss. Separating the two decisions makes the preparation stage reusable while keeping model selection tied to its output objective.

MAS4TS [46] combines decomposition, smoothing, statistical prediction, constraint projection, and residual correction. Its numerical operators illustrate ingredients of temporal processing; the router selects from a task-indexed chain registry using a downstream task objective. We therefore record the routed chain as a task-specific procedure, while its shared plot-anchor and latent-trajectory representation supplies a General Time-Series Harness module. TimeSage-MT [38] similarly provides a skill library that a planner uses across analytical turns. Atomic operators expose intermediate findings for inspection and revision; composed chains capture recurring sequences of analysis. The appropriate granularity depends on where new observations can change the plan. In either case, the next tool needs the processed window, applied transformation, and output scale. This makes explicit intermediate metadata a practical condition for reuse. Evaluation should check individual numerical results and the composed sequence, as in Section 7.

5.3. Temporal Context and Memory

Temporal analysis often requires information beyond the numerical sequence. Documents describe events, prior episodes provide analogies, and earlier analyses supply reusable procedures. The retrieval problem has two parts: representing relevance to the current signal and determining whether the evidence is available at the current analysis time. An Agent needs both to use retrieved material consistently.

NEXUS [47], Section 3.1, aligns historical values and contemporaneous textual factors into a chronological context timeline. The result preserves when a contextual explanation applies to the observations. This representation organizes the history around the temporal relationship between a numerical change and its accompanying evidence.

TIMECLAW [42], Section 3.4, retrieves prior analytical episodes through separate signal and text keys. A memory record is

$$m = (\mathcal{T}, \phi, \psi, \zeta, \rho), \tag{9}$$

where $\mathcal{T}$ denotes the task, ϕ and ψ are signal and text keys, ζ is the successful trajectory, and ρ records its transferable rationale. Similar wording may describe different signal structures, while similar shapes may occur under different contextual conditions. Separate keys preserve these two sources of relevance, and the retained trajectory explains how a preceding episode was solved. The combined record supports retrieval by signal structure and context, followed by reuse of the associated solution trajectory.

The task-specific boundary enters when the retrieval or retention policy depends on the target outcome. FinMem [26] ranks and promotes financial memories for trading, while FLAIRR-TS [10] combines historical-segment retrieval with observed forecasting errors. Their policies are analyzed under decision support and forecasting in Section 6. The memory design should follow the object to be reused: contextual events require temporal alignment, pattern knowledge requires signal-sensitive retrieval, and solution experience requires the associated trajectory and rationale. Task feedback then determines how those records are selected or retained. Comparing retrieved content, availability time, and downstream effects reveals whether the memory contributes relevant evidence to the current decision.

5.4. Temporal Validation

An analysis can execute successfully while using information from the wrong period or making a claim unsupported by its observations. Temporal validation checks these relationships before the task score is interpreted. AION [4] separates validity findings, including timestamp alignment and leakage boundaries, from scored outcomes such as forecasting error or report quality. The distinction allows the same temporal checks to precede several different output metrics.

Some implementations enforce availability at retrieval time. WorldReasoner [48] filters article and event records against the simulated forecast date. TIEM [49] additionally checks the provenance of retained skills: every ancestor must have resolved before the current cutoff. TimeSage-EV [50], Appendix A.6, imposes a related capability boundary by activating a newly accepted skill only in the next period. These mechanisms address different paths through which later information can enter an earlier analysis: retrieved records, inherited experience, and newly created tools. Equation 7 provides the common information boundary, while each implementation checks a particular access path.

Other checks concern transformations and claims. FactorMAD [51] instructs factor construction to exclude future timestamps and use rolling statistics instead of statistics computed over the full series. This check is separable from the financial value of the factor. TS-Agent [3] uses temporal question intents to declare required evidence predicates: an anomaly-location answer requires evidence of an anomaly and its supported segment, while other intents require trend, seasonality, or lag findings. The intent-specific gate is a Task-Specific Harness module: its required predicates and answer schema are defined by the temporal question. Shared numerical tools supply the findings that this QA procedure checks. The admissibility of the input and the sufficiency of the output evidence are complementary checks.

TSQAgent [52] evaluates data quality through dimension selection, evidence inspection, and pairwise adjudication. Its quality judgment can guide data selection before forecasting or classification. Across these examples, a validation interface should return the relevant finding and its supporting data, so that a developer can distinguish an invalid time boundary, a deficient input series, and a poorly supported answer. The placement of each check follows the failure it addresses: availability filters govern evidence access, data-quality judgments guide input selection, and evidence gates assess the support for a requested claim. These checks provide distinct feedback for revision. Task-specific scores then measure the usefulness of the accepted result. Together, the four temporal functions establish the data and evidence conditions under which the task procedures in the next section operate.

6. Task-Specific Time-Series Harness

Task-specific Harness modules organize shared capabilities around a required output and objective. This section follows the task axis of Figure 4: each family starts from its inputs and output contract, then compares the procedures used to construct, revise, and assess the result. The temporal data, processing, context, and validation functions in Section 5 supply reusable building blocks.

The task comparisons focus on how output requirements change tool selection, feedback, and stopping decisions. The four families organize the main task comparisons. Classification, imputation, annotation, and temporal question answering are also recorded in the task coordinate; the analytical operators in Section 5.2 and the question-answering evaluation in Section 7.3 connect to these additional outputs.

The task contract determines the final object being optimized. For forecasting, a typical contract is

$$\widehat{y}_{\tau+1:\tau+H} = f_\theta(\mathcal{I}_\tau; H), \tag{10}$$

where H is the declared horizon and $\mathcal{I}_\tau$ contains observed target history and covariates known at the origin, including future calendar information available under Eq. 7.

The four panels in Figure 8 connect each task output to its construction and evaluation. The illustrations show a forecast and its horizon, generated samples, a localized anomaly with diagnostic evidence, and alternative actions with their outcomes. Read each panel from the illustrated object to the input requirements, procedure, and evaluation criterion below it. The task output determines which feedback can guide revision and what counts as an acceptable result.

6.1. Forecasting and Prediction

Forecasting fixes an origin, target, horizon, and evaluation rule. The agent must choose how to build a forecast, which contextual evidence to use, and when to revise it. Numerical prediction produces future values or a predictive distribution; event prediction produces classes or probabilities for future outcomes. These contracts lead to different model interfaces and scoring rules.

Model selection and forecast construction. TimeSeriesScientist [23] develops a numerical forecasting workflow in which the Curator's diagnostics guide the Planner's model search. For each candidate forecaster, the Planner tunes its configuration using validation MAPE:

$$\theta_i^\star = \arg\min_{\theta_i \in \mathcal{C}_i} \mathrm{MAPE}_{\mathrm{val}}(m_i(\theta_i)), \qquad \mathrm{MAPE}_{\mathrm{val}}(m_i(\theta_i)) = \frac{100\%}{|\widetilde{D}_{\mathrm{val}}|} \sum_{t \in \widetilde{D}_{\mathrm{val}}} \left| \frac{x_t - \widehat{x}_t^{(i,\theta_i)}}{x_t} \right|. \tag{11}$$

The selected configurations are ranked, and the Forecaster uses their validation results to choose a single model or an ensemble, fixing its strategy and weights before evaluating the test set. This links the temporal diagnostics in Section 5 to a task-specific selection criterion. TimeCopilot [53] likewise makes forecasting models callable components; the model library supplies numerical capabilities while the agent organizes their use.

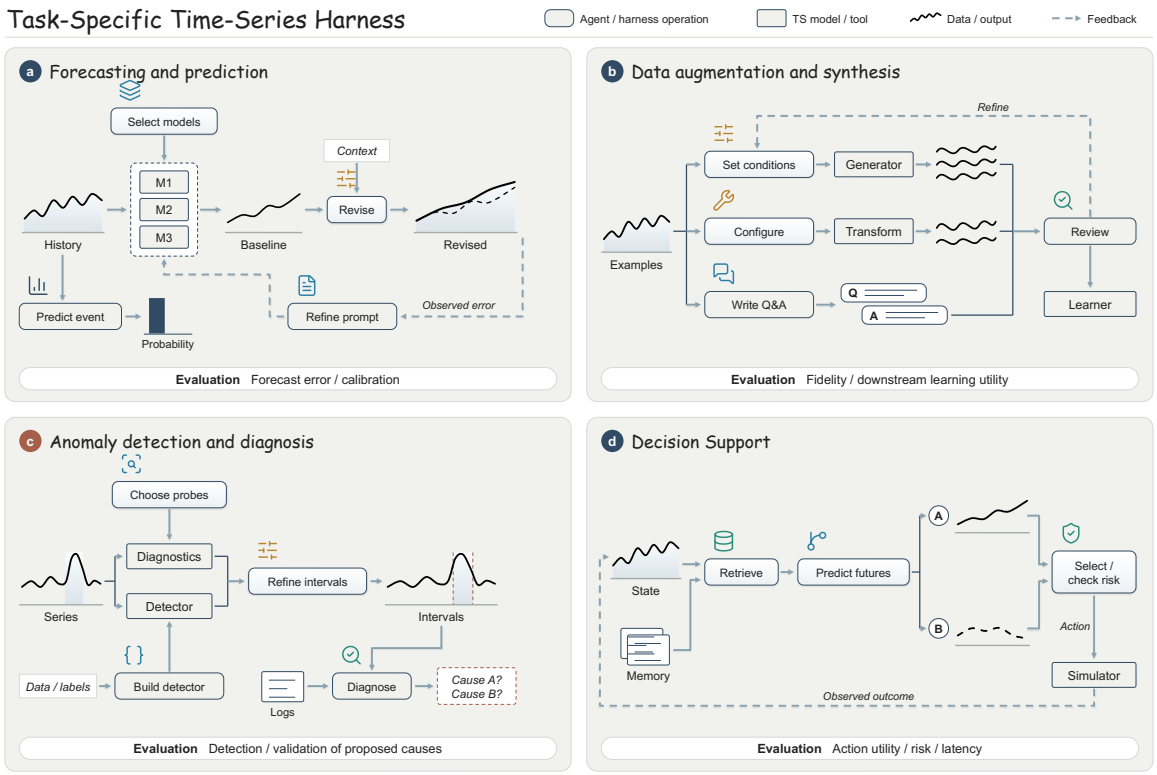


Figure 8. Task-specific procedures and their evaluation. Read each panel from the input series through model selection, generation, diagnosis, or action comparison to the resulting artifact. The return paths show feedback used to revise the procedure; the bottom of each panel identifies its evaluation target. Forecasting concerns numerical error and calibration, synthesis concerns fidelity and learning utility, diagnosis concerns detection and supporting evidence, and decision support concerns action utility and risk. These patterns synthesize the workflows discussed below, including TimeSeriesScientist [23], FLAIRR-TS [10], AnomaMind [11], and TradingAgents [37].

Contextual forecasting and revision. FLAIRR-TS [10], Algorithm 1, refines the forecasting prompt through retrieved historical segments and observed forecast errors:

$$\begin{aligned} S^{(k)}_{\text{retr}} &= \text{RetrieveSegments}(X_{\text{HistDB}}, X_{\text{Ctx}}, M), \\ C^{(k)}_{\text{aug}} &= \text{AugmentContext}(X_{\text{Ctx}}, S^{(k)}_{\text{retr}}), \\ \widehat{X}^{(k)} &= F_{\text{LLM}}(P^{(k)}_{\text{curr}}, C^{(k)}_{\text{aug}}, H), \\ \text{mae}^{(k)} &= \text{MAE}(\widehat{X}^{(k)}, X_V), \\ (P^{(k)}_{\text{next}}, d^{(k)}) &= R_{\text{LLM}}\Big(\{(P^{(i)}, \text{mae}^{(i)})\}_{i<k}, P^{(k)}_{\text{curr}}, \tau_{\text{stop}}\Big). \end{aligned} \tag{12}$$

Here X_V is the observed target sequence for an H-step refinement window $V = (t_1, \ldots, t_H)$. The refiner proposes prompt updates from earlier prompts and errors; early stopping returns the current prompt, while reaching the iteration limit returns the lowest-MAE prompt. In deployment, this feedback becomes available when the refinement window has resolved.

CastFSR [33] separates fast numerical forecasting, slower contextual deliberation, and reflective evaluation. The initial prediction supplies a numerical prior; retrieved context then supports a candidate revision. Its state and action families make that division explicit:

$$s_k = \Phi(I, M_k, \widehat{Y}^{\text{prior}}_k, C_k), \qquad a_k \sim \pi_\theta(\cdot \mid s_k), \quad a_k \in \mathcal{A}_{\text{fast}} \cup \mathcal{A}_{\text{slow}} \cup \mathcal{A}_{\text{reflect}} \cup \mathcal{A}_{\text{final}}. \tag{13}$$

$$
\begin{aligned}
C_k &= T_{\text{ctx}}(D_{\text{task}}, D_{\text{domain}}, X_{1:L}, H_{\text{ctx}}, \widehat{Y}_k^{\text{prior}}, M_k; W_k), \\
\widehat{Y}_k^{\text{cand}} &= G_\theta(\widehat{Y}_k^{\text{prior}}, C_k, M_k), \\
E_k &= E_\theta(\widehat{Y}_k^{\text{cand}}, \widehat{Y}_k^{\text{prior}}, C_k, M_k), \qquad \widehat{Y}_{1:H} = F_\theta(\widehat{Y}_k^{\text{cand}}, E_k, M_k).
\end{aligned} \tag{14}
$$

Here $I = (D_{\text{task}}, D_{\text{domain}}, X_{1:L}, H_{\text{ctx}})$ contains the target and horizon, domain description, observed series, and available contextual information; M_k stores the interaction history and W_k sets the context lookback. Reflection checks the candidate against temporal alignment, contextual support, and domain constraints such as units and capacity. Weak or redundant context can leave the numerical prior unchanged, whereas relevant evidence supports a targeted revision. Last-Mile Forecasting [54] and CastFlow [55] provide related settings for contextual revision, while HydroAgent [56] connects predictions to hydrological simulation and warnings.

SCENARIODIFF [57] uses a Historical Context Agent, a Scenario Agent, and an Anchor Guidance Agent to translate documents into historical summaries, a future scenario, and sparse anchor constraints. A diffusion forecaster generates the future trajectory conditioned on this information:

$$
\mathcal{L}(\theta) = \mathbb{E}_{x^{(0)},t,\epsilon}\left\| x^{(0)} - x_\theta\Big(x^{(t)}, t, c\Big)\right\|_2^2, \qquad x^{(t)} = \sqrt{\bar{\alpha}_t}x^{(0)} + \sqrt{1-\bar{\alpha}_t}\,\epsilon, \tag{15}
$$

Here $x^{(0)}$ is the future target horizon, t indexes diffusion steps, $\epsilon \sim \mathcal{N}(0, I)$, and c contains the numerical lookback, historical-text summary, and a scenario inferred from historical information. Anchor Blended Sampling edits anchor-relevant regions of the generated trajectory while retaining the remaining temporal structure. The generation step therefore implements probabilistic forecasting. It differs from the training-series generation below in both its output and its evaluation target.

Event and probability prediction. Event prediction specifies a future event label or probability as its output. TimeCAP [58], Section 3.3, first asks a frozen contextualizer to describe the input series, $s_x = A_C(x)$. A trained multimodal encoder then combines the series and its description to predict the event and construct a retrieval embedding:

$$
(\widehat{y}_{\text{MM}}, z) = E_\phi(x, s_x), \qquad \mathcal{S} = \{(s_{x_j}, y_j) : j \in \text{NN}_k(z)\}. \tag{16}
$$

The nearest neighbors are retrieved from the training set. Their descriptions and labels serve as in-context examples for the predictor LLM, which returns a discrete event class $\widehat{c}_{\text{LLM}}$. This class is converted to a one-hot vector and combined with the encoder logits $\widehat{y}_{\text{MM}}$, with $\lambda \in [0, 1]$:

$$
\widehat{c}_{\text{LLM}} = A_P(s_x, \mathcal{S}), \qquad \widehat{y} = \lambda\,\text{onehot}(\widehat{c}_{\text{LLM}}) + (1-\lambda)\widehat{y}_{\text{MM}}. \tag{17}
$$

The distinctive mechanism is the learned retrieval of relevant event examples and their use in complementary predictors. Here augmentation enriches the prediction context; the output contract is event prediction. AIA Forecaster [59] combines search, supervisor aggregation, and calibration for judgmental event forecasts; AUDITFORECAST [60] anchors predictions in a quantitative baseline and updates them in odds space. These methods require probability and outcome evaluation alongside evidence access and trace inspection. Their comparison is most informative when the target event, resolution time, and available evidence are fixed. Across forecasting workflows, a numerical trajectory calls for horizon-specific validation, while an event probability calls for an explicit resolution rule and calibration assessment. Retrieval, model selection, and forecast revision should be chosen according to the evidence available before that target resolves. This connects the Agent's analytical choices to the feedback that can actually guide them.

6.2. Data Augmentation and Synthesis

Augmentation and synthesis create examples for learning or analysis. The output may be a transformed positive view, an additional training series, or a temporal instruction dialogue. Its evaluation depends on that use: preserving a useful representation, training an accurate downstream model, and teaching a language model require different checks.

BRIDGE [61], Section 3, defines text-controlled generation through

$$\widehat{x} = f_\theta(\mathcal{D}, l, z), \tag{18}$$

where $\mathcal{D}$ supplies the domain, l describes desired temporal properties such as trends and periodicity, and z is sampled noise. Its multi-agent preparation stage collects textual templates, evaluates the resulting descriptions through a text-conditioned forecasting task, and refines the templates using feedback. The generation stage combines the description with domain prototypes to condition a diffusion model. This separates control of the generated object's properties from construction of the textual conditions used to obtain it. The forecasting evaluator supplies feedback for template refinement; the final generation experiments examine the synthetic series.

MERIT [25] targets unsupervised time-series representation learning. A retrieval agent selects relevant series from a candidate set; an augmentation agent uses those examples to choose transformations; and a review agent assesses the resulting positive views. Accepted samples enter a memory bank, while rejection sends the workflow back to retrieval for refinement. The retrieved neighbors and augmented views then support contrastive learning. The review acts on the generated examples during construction, while downstream experiments evaluate the learned representation. Figure 9 follows these decisions through a schematic augmentation example and shows how the resulting positive set enters the representation learner.

MetaCaster [39] generates training series from a small support set and domain context, then trains lightweight forecasters on the generated data. Its Harness Proposer changes the generator's prompts and skills to improve the downstream forecasting result. The optimization objective compares forecasters trained on authentic and generated data:

$$\theta^\star = \arg\min_\theta \; \mathbb{E}_{m,l}\, \delta\big(\omega(f_l^m), \omega(\bar{f}_l^m)\big), \qquad \bar{D}^m = \mathrm{MGAgent}(D^m_{\mathrm{sup}}, C^m), \tag{19}$$

$$\delta\big(\omega(f_l^m), \omega(\bar{f}_l^m)\big) = \max\left\{ \frac{\omega(\bar{f}_l^m) - \omega(f_l^m)}{\omega(f_l^m)}, 0 \right\} \tag{20}$$

Here θ denotes the generator's prompt and skills, and ω measures forecasting error on the evaluation split of a dataset in the harness-optimization corpus. The hinge-style discrepancy penalizes a generated-data forecaster when it performs worse than its authentic-data counterpart. This feedback updates the generator; final generalization is assessed on separate in-domain and out-of-domain datasets. After optimization, the Harness Proposer is removed and the learned prompts and skills guide generation for a new support set. Generation and harness optimization are coupled to the forecasting objective; the job scheduler and recovery mechanism remain separately reusable General Harness components. This provides a concrete example of one paper contributing distinct modules at different layers.

For language-model supervision, ChatAD [35] constructs temporal instruction dialogues through TSEvol; this model-side use of synthesis is discussed in Section 3. Comparing these methods requires identifying the generated object and the learner or analysis it is intended to support.

Figure 9 makes the design choice explicit: retrieval supplies examples, augmentation constructs positive views, and review determines which views enter contrastive learning. This workflow suits representation learning, where the generated views must preserve useful signal structure. MetaCaster

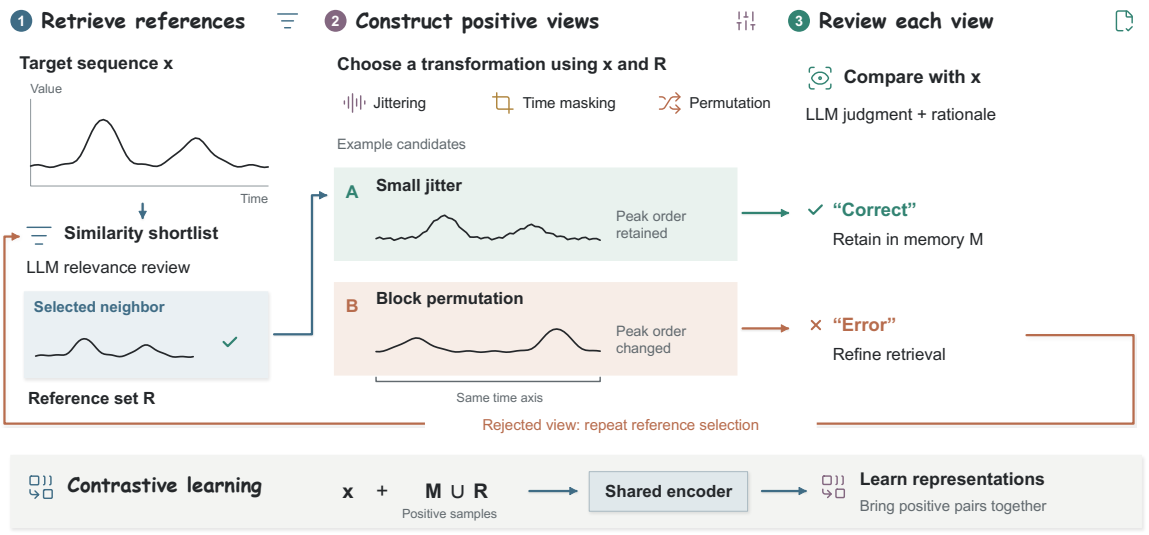


Figure 9. MERIT's construction of positive samples for representation learning [25]. Similarity retrieval and LLM relevance review produce a reference set R, which informs transformation selection. The schematic candidates contrast small jitter that preserves peak order with a permutation that changes it; these illustrate possible review outcomes. The review agent compares each view with the target sequence and returns a judgment and rationale. Accepted views enter memory M, while rejection restarts retrieval. The lower band shows the downstream role of these choices: accepted views and retrieved neighbors form positive samples for contrastive learning with a shared encoder. Adapted from the original retrieval, augmentation, review, and learning mechanisms.

couples generation to downstream forecast error, making forecaster performance the criterion for revising the generator. Selecting a synthesis workflow therefore starts with the intended learner and the feedback available to improve its training data.

6.3. Anomaly Detection and Diagnosis

Anomaly detection asks where a series departs from expected behavior; diagnosis adds an explanation of that departure and may lead to a response. These outputs require different evidence. An interval detector needs numerical and contextual support for the flagged region, while a service-diagnosis workflow also needs logs, incident context, and a mapping from symptoms to possible causes.

AnomaMind [11] develops the interval-detection workflow through coarse evidence acquisition, adaptive evidence construction, anomaly detection, and iterative refinement. At step k, the Agent maintains state s_k, selects action a_k, and receives numerical or knowledge evidence o_k. Its toolkit $B = \{K, N\}$ separates knowledge memory K from numerical diagnostics N. The workflow updates the evidence state and applies a learned detector to the accumulated numerical and contextual evidence:

$$s_{k+1} = E\Big(s_k, a_k, o_k, \widehat{Y}^{(k)}\Big), \qquad \widehat{y}_j = D_\theta\big(u_j, Z_{1:k}, C_{1:k}\big), \quad \widehat{y}_j \in \{0,1\}, \tag{21}$$

where $u_j = [\ell_j, r_j]$ is a candidate interval and $s_k = (S^{(k)}, Z_{1:k}, C_{1:k})$ contains the candidate set, numerical diagnostics, and contextual knowledge. During detector training, rule-based feedback combines parse validity, agreement with labeled anomalous intervals, and a false-positive penalty:

$$r_k = \lambda_p r_k^{\text{parse}} + \lambda_f r_k^{F1} - \lambda_{fp} r_k^{FP}, \qquad \max_\theta \ \mathbb{E}_{\widehat{Y}^{(k)} \sim \pi_\theta(\cdot|s_k)} \left[\sum_{k=1}^{K} r_k \right]. \tag{22}$$

The reward supplies learning signals for a parseable and accurate interval decision. Figure 10 follows the four stages and shows how evidence acquisition and iterative revision support that output.

The trained detector belongs to the LLM side; the surrounding evidence-acquisition and refinement procedure supplies the anomaly-specific Harness.

SAGE-AD [24] uses a complementary form of evidence. It preserves the original series for precise tools and a compressed representation for the LLM. Family-specific analyzers and diagnostic visualizations feed a detector that returns anomaly intervals, confidence, and anomaly types. Its in-context reference database is constructed from normal training segments: representative subsequences are selected by clustering, then rule-based perturbations create normal–anomalous examples for nine anomaly types. This makes example retrieval part of the detection procedure, with the family-specific evidence guiding the interpretation of a suspicious interval.

Detection rules and executable pipelines. ARGOS [62], Sections 3.1–3.3, turns labeled temporal chunks into executable detection rules. A Detection Agent proposes rules, a Repair Agent executes them on dummy inputs to resolve implementation errors, and a Review Agent compares their accuracy on validation data. During deployment, the rules are combined with an existing detector. The learned artifact is a reusable detector program, whereas AnomaMind selects evidence and refines an interval decision within an analysis. Their evaluation must therefore account for when rule construction occurs and which labels are available during that phase.

AD-AGENT [63] constructs a pipeline from a natural-language request and a dataset. Its Processor extracts modality and supervision requirements, the Selector chooses a supported library and model, and the subsequent agents retrieve implementation information, generate code, and review execution. TSLib supplies its time-series branch. This approach suits requests that require assembling an executable detector from existing libraries. Its Reviewer tests execution on synthetic samples; optional Evaluator and Optimizer agents assess labeled data and tune hyperparameters. For a deployed time-series pipeline, executable-code success and detection performance on held-out series provide complementary checks. Across these methods, the central choice is whether to learn evidence acquisition, retrieve diagnostic examples, or construct a detection pipeline; the available supervision, evidence sources, and required output determine which procedure fits the scenario.

From detection to diagnosis. LEMAD [44] extends the output to service diagnosis in power-grid operations. Metric and log collectors send aligned streams through Kafka to a real-time analysis agent, which correlates the signals and produces alert summaries and preliminary hypotheses. An anomaly-detection agent further evaluates suspicious events using hybrid models and contextual reasoning, returning confidence, root-cause localization, and recommendations. Its reported evaluation includes precision, recall, F1, and latency on service data. The construction lesson is that adding diagnosis changes the evidence inputs and final artifact: an interval label becomes one input to a broader incident-analysis workflow.

6.4. Decision Support

Decision-support Agents select an action using predictive or diagnostic evidence. The action space and feedback determine the task-specific procedure: portfolio allocation acts through positions and returns, traffic control acts through signal configurations and queues, and maintenance support produces recommendations for review. The Agent must therefore represent both the temporal state and the consequences of the available actions.

The context policy can itself be specialized for an action. FinMem [26], Section 3, separates financial memories into layers with different retention times and ranks events using recency, relevance, and importance. Significant investment memories can be promoted to deeper layers, and retrieved events condition a trading decision together with the current market state. The memory policy therefore depends on the useful lifetime of financial information and trading experience. A general retrieval component supplies storage and access; this task-specific policy decides what to retain and how it influences the action.

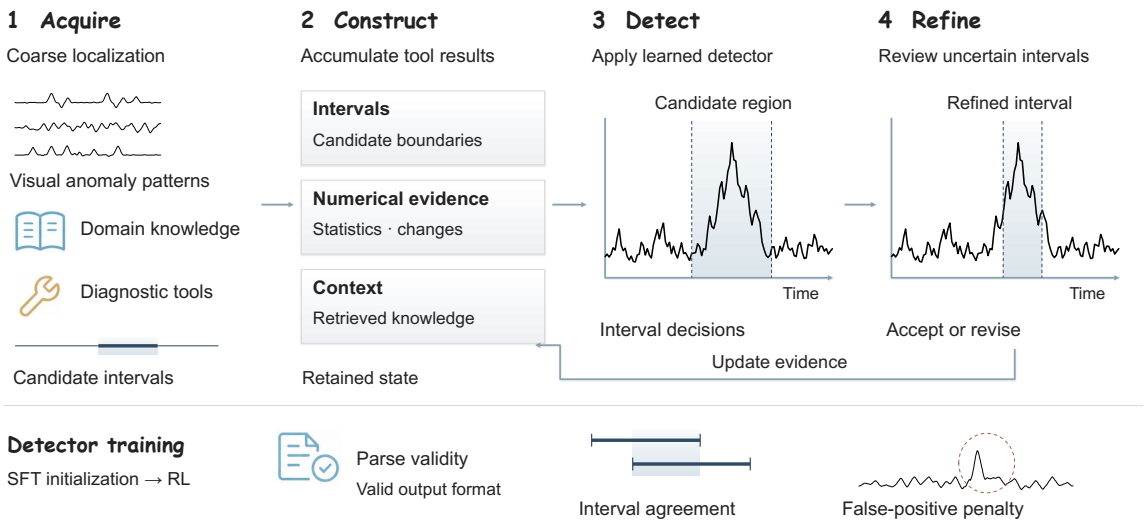


Figure 10. AnomaMind's evidence acquisition, state construction, interval detection, and refinement [11]. Read the upper sequence from knowledge and numerical diagnostics to the evidence state and candidate intervals. The paired plots use the same schematic signal to show how refinement changes the interval boundaries; the return path updates the evidence state. The separate lower band identifies detector-training signals: parse validity, agreement with labeled intervals, and a false-positive penalty. Adapted from the method's workflow.

FinCon [41] specifies financial analyst roles that deliver investment insights to a trading manager. Their output contracts include market evidence, stock selection, and risk feedback, making the implemented coordination procedure task-specific. Its prompt-controlled policy maximizes cumulative profit and loss,

$$\max_{\theta} \mathbb{E}\left[\sum_{t\in\mathcal{T}} \alpha^t R_t^{\Pi_\theta}\right], \qquad R_t^{\Pi_\theta} = R(O_t, B_t, A_t), \tag{23}$$

where θ comprises agent prompts, α discounts future rewards, and O_t, B_t, and A_t denote market observations, managerial reflection, and trading action. FinCon updates prompts between training episodes using comparative performance feedback; within-episode risk alerts also guide testing decisions. Its portfolio manager solves a mean–variance allocation subject to the buy/sell/hold direction selected by the agents:

$$\max_{w} \langle w, \mu\rangle - \langle w, \Sigma w\rangle, \qquad w_n \in \begin{cases} [0,1], & \text{buy,} \\ [-1,0], & \text{sell,} \\ \{0\}, & \text{hold,} \end{cases} \quad n = 1,\ldots,N. \tag{24}$$

Here μ and Σ are shrinkage estimates of mean daily returns and their covariance. In FinCon's encoding, w_n is portfolio exposure: buy permits a long position, sell permits a short position, and hold assigns zero exposure. The portfolio is rebalanced daily. The analyst–manager interaction supplies the directional evidence, and the allocation problem converts that evidence into a position under the method's risk objective. TradingAgents [37] emphasizes a different decision procedure: analyst reports feed bullish and bearish research roles, whose debate precedes trading and risk deliberation. Both systems connect evidence aggregation to an action, while their coordination and acceptance procedures differ.

CoLLMLight [2] makes the prediction-to-action relation explicit in traffic control. Each intersection observes queue lengths, moving vehicles, waiting time, and occupancy at its own and neighboring

lanes. A graph G represents their connectivity, and a history window preserves the observations and signal configurations that preceded the current decision:

$$\mathcal{T}_t = \{(O_{t_i}, a_{t_i}) : t - \Delta t < t_i < t\}, \qquad X = \text{Prompt}(O_t, G, \mathcal{T}_t, D). \tag{25}$$

For complex coordination, the LLM analyzes current traffic, predicts neighboring states under candidate configurations, and selects a signal:

$$Y_{\text{ana}}, \{\widehat{O}^a_{t+1} : a \in \mathcal{A}\}, \widehat{a} = f_{\text{LLM}}(X). \tag{26}$$

Here D supplies the control instructions. Simpler situations omit some reasoning stages according to the number of critical neighboring lanes; simulation-derived data support policy fine-tuning. The resulting action changes future traffic observations, so evaluation measures policy outcomes in the control environment. Portfolio and traffic applications both require reasoning about future outcomes; each instantiates its own state, feasible actions, feedback timing, and utility. A decision-support Harness must connect these quantities to an executable policy and measure the consequences of that policy over time.

A new decision-support Agent should specify these quantities before choosing its reasoning workflow. The numerical or language prediction supplies evidence; the action contract determines how that evidence becomes a position, a control signal, or a recommendation. Section 7 connects each resulting artifact to an appropriate evaluation protocol.

7. Infrastructure and Benchmarks for Time-Series Agents

Evaluation completes the construction of a Time-Series Agent: the benchmark must expose the information, actions, and output requirements that the deployed workflow will encounter. AION [4], Section 3.1, expresses this connection through a task file ϕ, an accessible workspace $\mathcal{W}$, and a validator $\mathcal{V}$:

$$\mathcal{T} = (\phi, \mathcal{W}, \mathcal{V}), \qquad \phi = (\mathcal{T}_{\text{access}}, \Sigma, \mathcal{O}, \mathcal{C}, \mathcal{A}), \qquad (g, s) = \mathcal{V}(o). \tag{27}$$

The task file specifies access, prior knowledge, output form, constraints, and metrics. The validator returns validity checks g and scores s for candidate output o. This separation lets an evaluator identify both an incorrectly executed analysis and a valid analysis with poor predictive performance. The reviewed benchmarks instantiate these requirements in different ways: some grade submitted code, others test evidence discovery, and others evaluate a sequence of analytical turns or recurring data releases.

7.1. Evaluation contract

The unit of evaluation is an episode with a declared information boundary. Its record includes the request, visible data, tool calls, intermediate artifacts, validator decisions, revisions, final output, and resource use. Three questions connect this record to the construction framework. Did the General Harness execute the intended operations and handle failures? Did the temporal modules preserve the data window, numerical meaning, and information availability? Did the task module produce an accurate forecast, useful synthetic sample, supported diagnosis, or effective decision?

These questions require different observations. A tool-call schema check establishes that a requested operation is well formed; running the tool establishes that it executes; comparing its output against a reference establishes numerical correctness. TSAG [64], for example, evaluates invocation over hard-coded tool stubs. That protocol isolates tool selection and argument formation. TimeSeriesGym [22] instead evaluates artifacts produced in an executable environment, so errors in code, data handling,

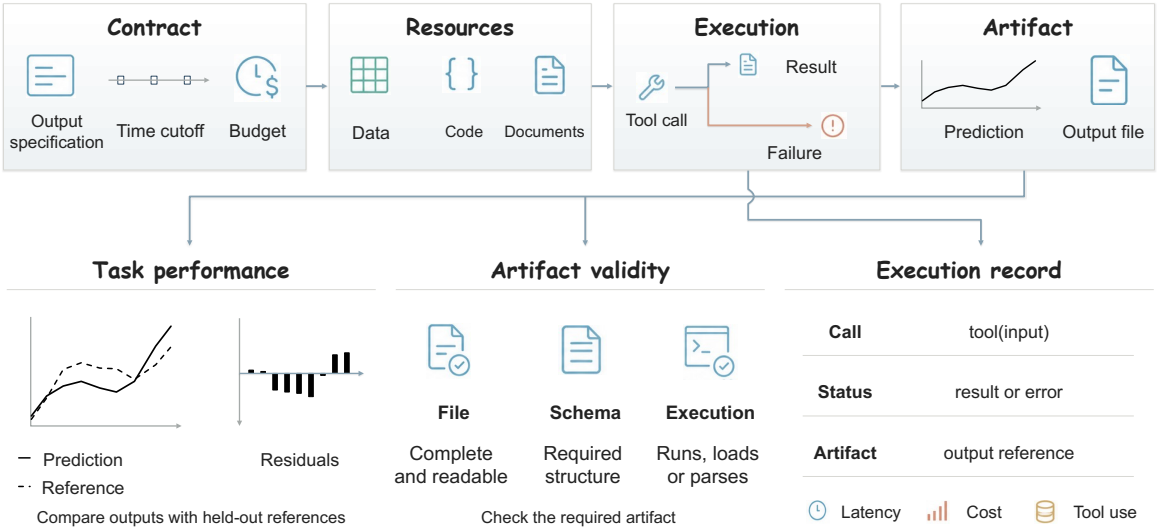


Figure 11. An evaluation episode and three complementary forms of evidence, synthesized from AION [4] and TimeSeriesGym [22]. The upper sequence connects the contract, resources, execution, and submitted artifact. The lower views distinguish task performance through prediction and residual plots, artifact validity through file, schema, and execution checks, and the execution record through calls, status, artifact references, and resource use. Evidence links connect each assessment to the object it examines, allowing poor predictions, invalid submissions, and execution failures to be diagnosed separately.

and submission format affect the outcome. The choice between these protocols follows the capability being tested.

Figure 11 follows an episode from its task contract and accessible resources through execution to the submitted artifact, and separates the three records needed to interpret its outcome. An inaccurate forecast can arise from a weak predictor, a failed tool call, or incorrectly aligned input; preserving the execution trace distinguishes these causes. Resource measurements add the cost of reaching the result and of recovering from failures.

7.2. Existing benchmark landscape

TimeSeriesGym [22] evaluates time-series ML engineering through 34 challenges from 23 sources, spanning eight temporal problem types. A challenge combines resources such as data, code, papers, and documentation with a task description and a grading function. The Agent must produce a usable artifact within the execution budget. The reported protocol uses a Docker environment and default limits of four hours and 50 steps, and distinguishes valid and reasonable submissions from task-native performance. This makes it suitable for a system expected to inspect a repository, adapt a model, and deliver runnable analysis. Figure 12 summarizes the resources-to-artifacts path; the grader closes the loop between the requested engineering task and the submitted result.

TimeSage-MT [38] examines multi-turn analytical continuity. Its 240 tasks span four levels and eight domains; each task includes a visible series slice, dialogue, analysis trajectory, reference code, numerical ground truth, and output requirements. Outcome scoring measures whether the requested result is correct, while capability diagnostics examine the steps used to obtain it. Its reference system combines an LLM Planner and Reporter with deterministic validation, profiling, execution, and evaluation, and exposes 226 analytical skills across 36 categories. It therefore supports comparisons between model backbones and between runtime configurations under the same temporal workflow.

TimeSage-EV [50] adds a second timescale: the analysis recurs as new evidence becomes available. The reported snapshot contains 60 scenarios across six domains and 1,485 scenario-period instances

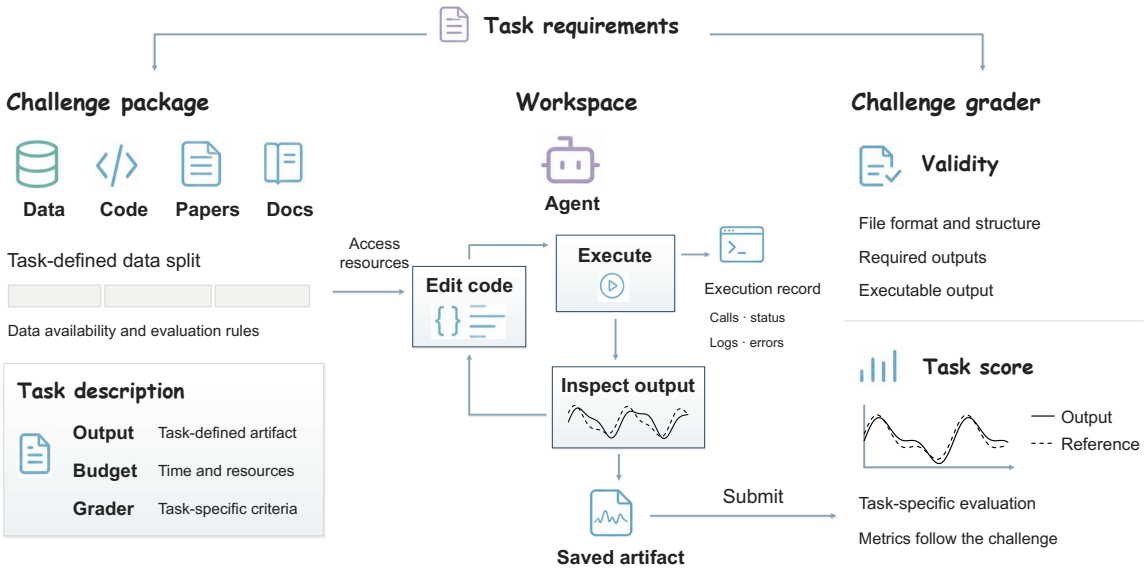


Figure 12. Executable evaluation in TimeSeriesGym [22]. Challenge resources and instructions enter an execution environment in which an Agent produces code, models, or other outputs. A challenge-specific grader checks the submitted artifact and its task metrics. The shared challenge contract enables comparison across Agents; the workspace shows how working files, tool execution, and saved outputs connect the challenge package to its grader. Adapted from the benchmark's evaluation structure.

spanning February 2023 to May 2026. At each period the Agent receives the permitted evidence and produces structured answers and reports, while the next target release is withheld for evaluation. The benchmark measures keypoint accuracy and coverage together with report faithfulness and quality, and compares independent with sequential contexts. A system intended for recurring economic or operational monitoring can thus be tested for its use of accumulated context as well as its current answer.

Contextual forecasting benchmarks isolate the value of external information. Dr-CiK [8] provides 240 forecasting tasks and 8,849 documents with supporting evidence, multi-hop dependencies, and distractors. Its evaluation separates research and evidence recovery from forecast quality, allowing a designer to investigate whether a failure occurred during retrieval or after the relevant context was found. TemporalBench [65] varies historical and future partitions and the availability of contextual or event information across four temporal understanding and forecasting settings. These controlled conditions help test when context changes a numerical or language-facing prediction.

TFRBench [66] evaluates forecasting explanations as well as numerical forecasts on ten datasets across five domains. Its reference construction uses future events and outcomes to select explanatory trajectories; test-time systems receive historical inputs. An LLM judge compares candidate reasoning with those references along domain relevance, forecasting correctness, event relevance, and logic-to-number consistency, alongside numerical metrics such as MAE and MASE. The resulting signal measures agreement with outcome-conditioned reference reasoning. For deployment-oriented evaluation, that signal can be paired with an audit of the information actually available when the forecast was issued.

TraceBench [67] studies root-cause attribution through controlled dynamical-system interventions. An Agent identifies which simulator parameter changed, with conditions varying noise, domain context, support examples, and direct versus programmatic submission. A submitted program can also be evaluated on held-out samples. The benchmark records accuracy together with tokens, cost, elapsed time, tool use, Python execution, and plotting behavior. It therefore exposes the trade-off between exploratory analysis and diagnostic generalization. TSQBench [52] addresses a different

prerequisite: selecting trustworthy temporal data. Controlled pairs are derived from a common template and degraded along specified quality dimensions, enabling comparisons of quality identification, pairwise judgment, and downstream data-selection utility.

Table 4 condenses these differences into the output the Agent must produce and the evidence the evaluator observes. The rows support concrete selection decisions: engineering artifacts call for an execution environment; multi-turn analysis calls for continuity checks; contextual forecasts call for evidence controls; and diagnosis calls for interventions with known causes.

Resource	Output	Evidence
TimeSeriesGym [22]	Runnable ML artifacts	Submission validity, task score, execution budget.
TimeSage-MT [38]	Multi-turn analytical results	Numerical outcomes, intermediate capabilities, reference trajectories.
TimeSage-EV [50]	Recurring answers and reports	Release cutoffs, accumulated context, keypoints and report quality.
Dr-CiK [8]	Context-informed forecasts	Supporting-document recovery, multi-hop evidence, forecast quality.
TemporalBench [65]	Temporal answers and forecasts	Historical/future partitions and controlled contextual information.
TFRBench [66]	Forecasts with reasoning	Numerical error and outcome-conditioned reference reasoning.
TraceBench [67]	Root causes or diagnostic code	Known simulator interventions, held-out accuracy, exploration cost.
TSQBench [52]	Data-quality judgments	Controlled degradations, dimension-wise comparison, selection utility.

Table 4. Benchmark selection by required output and observable evidence. Each resource tests a particular part of the Agent workflow; the surrounding text describes its protocol and interpretation.

7.3. Temporal question answering

Question answering tests whether a model or Agent can interpret temporal evidence in language. TSAQA [68] covers six analytical tasks, including anomaly detection, characterization, comparison, and temporal relationships, using true/false, multiple-choice, and puzzling formats across thirteen domains. MMTS-Bench [69] contains 2,424 question–answer pairs in Base, InWild, Match, and Align subsets, testing structural perception, temporal reasoning, sequence matching, and cross-modal alignment. Such evaluations are useful when a user asks about a trend, a change, or an event visible in a series. In an operational Agent, the same question can initiate a tool workflow: inspect the series, calculate a statistic, retrieve relevant context, and return an answer tied to those observations.

TS-Agent [3] illustrates this connection through analytical operations and evidence-conditioned answers. Figure 13 instantiates the method's anomaly-location question: the evidence must establish both that an anomaly exists and which part of the series contains it. The numerical observation fills these requirements in the evidence log; an uncovered requirement directs the next action. Inspecting this trace reveals how the answer follows from the available observations. The appropriate extension depends on the downstream use: a forecast adds a horizon and numerical scoring rule, while diagnosis adds event localization and cause verification. QA then serves as one measurable capability within that larger task.

7.4. Infrastructure for reproducible evaluation

Infrastructure determines which parts of an episode can be replayed. AION [4] specifies task files, workspaces, artifacts, and validation interfaces. TimeSeriesGym [22] packages challenge resources and graders into executable environments. TIMECLAW [42] keeps numerical series in a workspace and exposes executable analytical tools, trajectories, and memory. These choices make different boundaries explicit: the task request, the execution environment, and the persistent analytical state.

ModernTSF supplies a forecasting infrastructure case around callable model and dataset resources and reproducible runs. In a new Agent system, such a substrate can supply the numerical tool layer, while the Agent adds evidence retrieval, model selection, revision, and reporting. Recording the dataset version, model configuration, tool inputs and outputs, and resulting artifacts makes these choices comparable across runs. The comparison is especially useful when the same numerical predictor is embedded in several Harness designs.

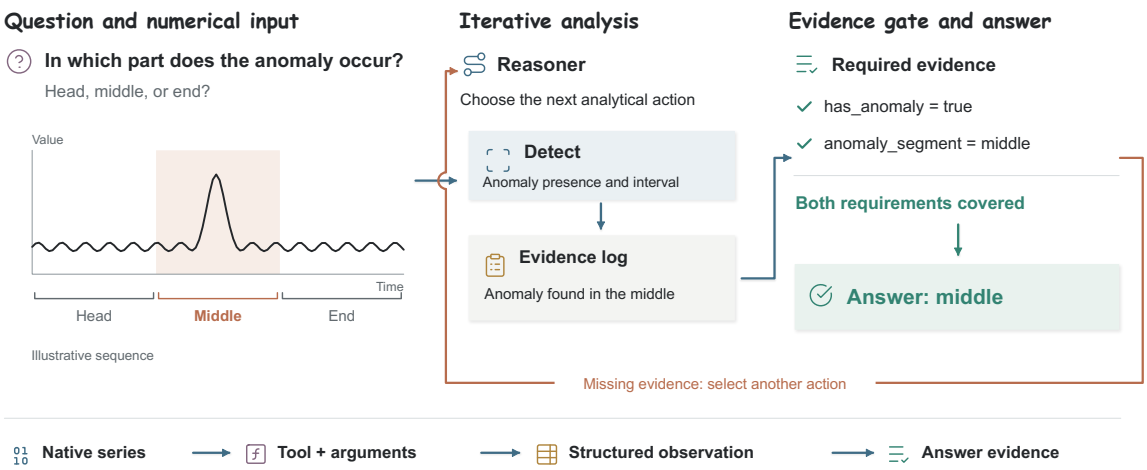


Figure 13. An evidence-conditioned answer in TS-Agent [3]. The anomaly-location question and its required predicates follow the original method; the sequence and middle-region outcome are illustrative. A numerical tool analyzes the native series and records an observation. The evidence gate checks both anomaly presence and location before accepting the answer; an uncovered requirement triggers another analytical action. The lower chain identifies the records needed to trace the answer to the series, tool invocation, and structured observation. This example shows how temporal question answering can evaluate evidence acquisition and answer verification within a larger Agent workflow.

TIMECLAW [42] in Figure 14 connects the numerical workspace to temporal tools and multimodal memory. The diagram shows where the series is stored, how analytical operations execute, and how past analytical experience becomes accessible. The lower path shows how recurring executed procedures become candidate tools, which are evaluated on held-out similar tasks before registration. Reproducing a run therefore requires the workspace and memory state alongside the model and task inputs. A separate TimeClaw exploratory-learning method [7] compares executed trajectories and distills experience; its update equations appear in the focused analysis below.

7.5. Selecting a benchmark for a new scenario

Resource selection follows the same task–function grid as system construction. Begin with the required artifact and the mechanism whose contribution is being tested. A numerical model library supplies callable forecasting capabilities; an execution environment establishes whether the Agent can deliver a working analysis; a controlled context benchmark measures how external evidence changes the result. Each answers a different development or evaluation question.

For the demand-planning example in Figure 4, ModernTSF supplies model and dataset routines for the numerical tool layer. A direct forecasting run provides a baseline for the Agent's selection or revision procedure. Dr-CiK [8] provides a controlled test of evidence retrieval and contextual prediction, including supporting and distracting documents. TimeSeriesGym [22] tests whether the Agent can construct the required executable artifacts under a challenge's grading protocol. An inventory simulator then evaluates how the forecasts affect stockouts, holding costs, and feasible actions under the chosen operational assumptions. These evaluations isolate numerical prediction, evidence use, execution, and downstream utility within the same proposed workflow.

Other task contracts lead to different combinations. TimeSage-MT [38] tests continuity across analytical turns, and TimeSage-EV [50] tests revision and capability reuse across evidence releases. TraceBench [67] supplies interventions with known causes for diagnostic exploration. TSQBench [52] tests dimension-specific data-quality comparisons before downstream analysis. Selecting a resource by the capability it exposes makes the result interpretable: a quality judgment, a runnable program, a forecast, and a diagnosis are distinct artifacts with distinct evidence requirements.

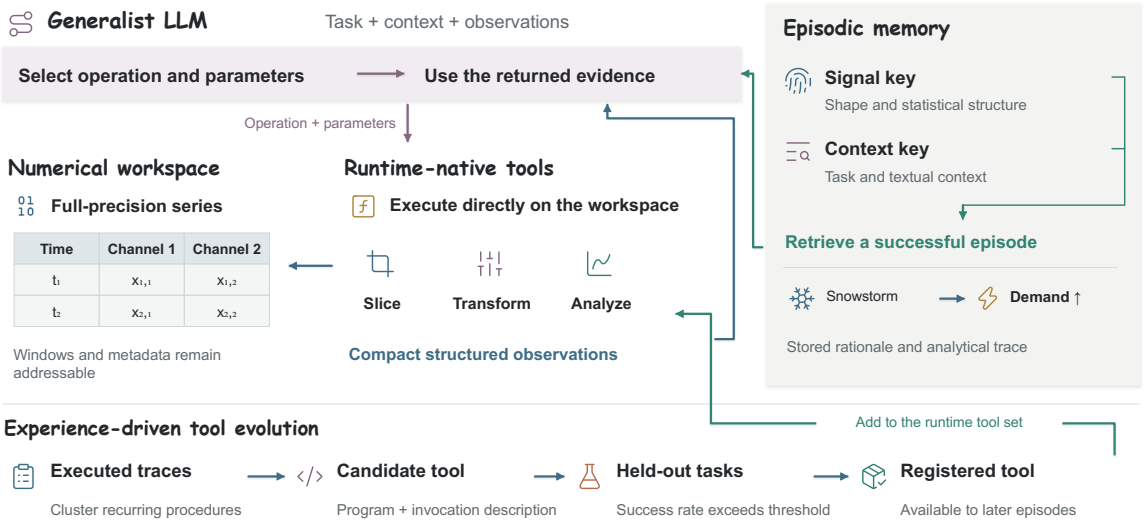


Figure 14. TIMECLAW's runtime and capability evolution [42]. The generalist LLM selects an operation and its parameters; temporal tools execute on a full-precision numerical workspace and return compact observations. Episodic memory retrieves successful analytical traces using signal and contextual keys, together with a transferable rationale. The lower path converts recurring executed procedures into candidate tools, evaluates them on held-out similar tasks, and registers accepted tools for later episodes. The two paths identify the workspace, memory, and tool-set state needed to reproduce the system. Adapted from the original workspace, tool, memory, and evolution mechanisms.

Augmentation and decision support require evaluation through the use of the resulting data or actions. MERIT [25] evaluates representations learned from generated positive views on downstream temporal tasks, whereas MetaCaster [39] evaluates forecasters trained on synthetic series through forecasting error. The intended learner therefore determines the synthesis evaluation, with separate data for revising the generator and assessing its result. FinCon [41] evaluates trading and portfolio decisions using cumulative return, Sharpe ratio, and maximum drawdown; CoLLMLight [2] evaluates traffic-signal decisions in CityFlow through average travel and waiting times. For a new application, these examples guide the choice of action space, outcome measure, and feedback schedule. An inventory workflow would similarly require an environment specifying demand, lead times, feasible actions, and costs, alongside its forecasting tests.

A component comparison should preserve the backbone, accessible tools, data boundary, and resource budget while changing the module under investigation. The record should connect the input, intermediate artifacts, checks, and final output, so that an improvement can be attributed to processing, retrieval, validation, or the task procedure. A repository supplies the means of implementing the comparison; a benchmark supplies a declared protocol for judging it.

8. Open Problems and Focused System Analyses

8.1. Reusable temporal infrastructure

The preceding comparisons suggest a research priority: make temporal operations reusable through explicit inputs, state, and validation, then test the resulting compositions under changing task requirements. The same numerical operations recur in otherwise different workflows: a forecaster inspects missing values before fitting a model, a diagnostic Agent decomposes a signal before localizing an event, and an analytical assistant aligns observations before comparing periods. TIMECLAW [42] and the skill library in TimeSage-MT [38] provide concrete shared tool layers. A next step is to evaluate the transfer of individual tools under changes in task, sampling frequency, units, and data availabil-

ity. The module-level organization makes this comparison possible because each operation retains its input, output, scope, and source evidence.

A shared tool also needs a stable record of what it operated on. AION [4] connects the task file, workspace, and validator; TimeSage-EV [50] makes evidence release time part of the recurring analysis. Together these designs motivate interfaces that carry the observation window, data version, and admissible context through execution. A practical test is to replay an earlier analysis after a later release has arrived and verify that the original evidence boundary is preserved.

8.2. Open problems

Connecting temporal perception to executable analysis. ChatTS [20] learns to interpret temporal attributes and language instructions, while tool-based systems such as TS-Agent [3] explicitly compute analytical evidence. Their combination raises a concrete allocation problem: which judgments should be handled by learned temporal perception, and which should trigger a numerical tool? Matched-backbone experiments can vary the tool policy and compare numerical error, evidence quality, and execution cost. This would identify where model adaptation reduces tool dependence and where explicit computation remains valuable.

Memory under changing information. TimeSage-MT [38] tests continuity within a dialogue, whereas TimeSage-EV [50] tests recurring analysis as releases accumulate. These settings distinguish remembering a useful result from deciding whether it is still applicable. Future memory evaluations can preserve an old explanation while changing its supporting observation, release vintage, or regime, then test whether the Agent revalidates, revises, or retires that explanation. This links memory quality to the temporal validity of later actions.

Transferring learned procedures. TIMECLAW [42] promotes generated tools using held-out execution, and TimeClaw [7] distills successful trajectories into external experience. The remaining question is how far each acquired capability transfers. Evaluation can hold out a task family, a domain, or a temporal regime while keeping the base LLM fixed. Recording tool identity, training episodes, and promotion criteria makes it possible to distinguish reusable procedure learning from repeated adaptation to similar instances. TimeSage-EV [50] provides a concrete availability rule through next-period skill activation.

Evidence-sensitive evaluation. Dr-CiK [8] controls supporting and distracting documents, TraceBench [67] controls diagnostic interventions, and TFRBench [66] scores reasoning against outcome-conditioned references. These protocols address different relationships between evidence and result. Combining controlled evidence changes with an executable trace can test whether a correct answer was supported by information available to the Agent and whether removing a relevant observation changes the conclusion appropriately. Such tests would make reasoning evaluation more directly connected to the operations that produced the answer.

Closing the prediction-to-decision loop. Forecast quality matters through its use. FinCon [41] couples analytical evidence to portfolio positions, and CoLLMLight [2] connects local traffic observations to network control. These settings introduce action costs, risk, delayed feedback, and effects on subsequent observations. Evaluations should compare the complete decision procedure under the same information and action constraints, recording both predictive performance and the resulting utility. A method that changes recommendations also needs evaluation across repeated decisions, where its own actions can alter the future data distribution.

Figure 15 connects these questions to the system interfaces they affect. Read each of the five rows from the open question to the controlled comparison and the evidence to record. Changing the information release, task setting, evidence, or output criterion exposes a different aspect of reuse and reliability. The three design analyses below isolate complementary choices along that path: the numerical input interface, retained execution experience, and the forecasting substrate.

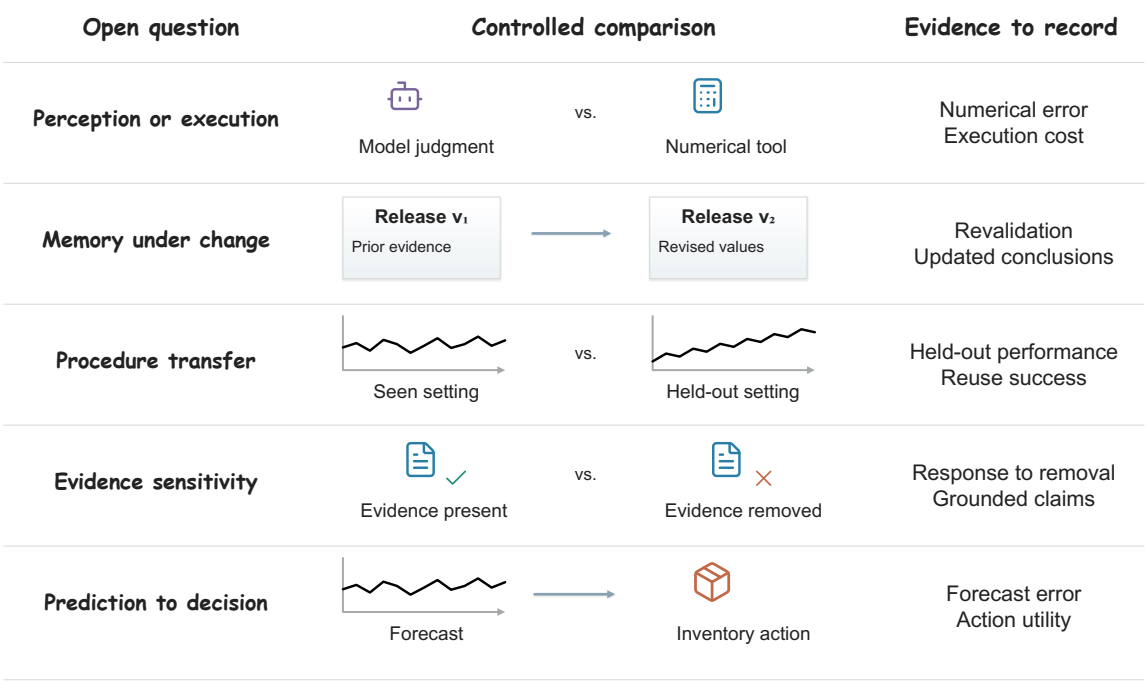


Figure 15. Controlled comparisons for reusable Time-Series Agents. The five rows pair an open problem with a testable contrast: model judgment and tool use, old and revised evidence, seen and held-out settings, supporting evidence and its removal, and forecast quality and downstream action utility. The paired objects show the intervention: a tool call, a revised release, a held-out series, evidence removal, or an inventory action. The right column identifies the outcomes to examine. Matching the backbone and resource budget helps isolate the factor under study. This agenda synthesizes temporal interfaces [20], evolving execution [42,50], controlled evidence [8], and decision workflows [41].

8.3. Design implications

ChatTS: selecting a temporal interface. ChatTS [20] makes the numerical input interface a design variable. An analytical assistant can compare direct text serialization with a learned temporal encoder while keeping its runtime, tools, and task inputs fixed. Preserving scale metadata in both conditions allows the comparison to examine magnitude-sensitive questions as well as temporal shape. Answer quality, evidence grounding, and tool use then reveal how the input representation changes the analysis.

TimeClaw: learning from executed trajectories. TimeClaw [7] develops an Explore–Compare–Distill–Reinject loop with a fixed underlying LLM. Alternative tool trajectories are executed, scored, and distilled into external experience. For task instance x, the method selects the best valid execution using the task loss:

$$q(\pi_k; x) = -L_\tau(\hat{y}_k, y^\star), \qquad \pi^\star(x) = \arg\max_{\pi_k \in \Pi_{\text{valid}}(x)} q(\pi_k; x). \tag{28}$$

Here π_k denotes an executed candidate trajectory, L_τ the loss for task type τ, and $y^\star$ the exploration target. The scored comparison occurs during experience construction; later inference receives the retained experience. The comparison is summarized and cleaned, and the resulting evidence updates the task-scoped experience state A_s:

$$\begin{aligned} n_x &= \text{Summarize}\Big(x, \Pi(x), \{q(\pi_k; x)\}_{k=1}^{K}, \pi^\star(x)\Big), \\ e_x &= \text{Clean}(n_x), \qquad A_s^{(m+1)} = \text{Distill}\Big(A_s^{(m)}, e_x\Big). \end{aligned} \tag{29}$$

The experience-update mechanism can use different task losses and is recorded as General Harness. Individual analytical operations retain their temporal or task-specific scopes. Comparing the initial and updated experience states on held-out episodes, with the backbone and tool access fixed, measures the contribution of retained experience. Separate tests across new instances, domains, and task types establish the scope over which that content remains useful.

ModernTSF: connecting numerical models to an Agent. ModernTSF[4] can hold the numerical forecasting substrate constant while the Agent workflow changes. A direct forecasting run and an Agent-controlled run can share the same model routines, data windows, and computational budget; the latter adds model selection or revision. Recording the chosen configurations, validation outcomes, and final errors makes the comparison informative about the selection procedure and its execution cost.

9. Conclusion

This survey explains how to build Time-Series Agents from general-purpose capabilities through a module-level taxonomy of General Harness, General Time-Series Harness, and Task-Specific Harness. These scopes distinguish reusable execution mechanisms, shared temporal functionality, and procedures tied to a task objective, while the LLM connects requests, tool actions, and observations. Comparing methods through their mechanisms, assumptions, and feedback clarifies which components a new scenario can reuse and which require adaptation. Linking these choices to development resources and evaluation protocols provides a construction path from scenario requirements to tested systems, spanning forecasting, synthesis, anomaly diagnosis, and decision support. The same framework motivates research on transferable temporal procedures, experience under changing regimes, and evaluation of downstream actions. The accompanying structured literature records support continued comparison as new modules and tasks emerge.

[4] ModernTSF repository: https://github.com/Diaugeia/ModernTSF.

References

1. Lim, B.; Zohren, S. Time-Series Forecasting With Deep Learning: A Survey. *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences* **2021**, [2004.13408]. https://doi.org/10.1098/rsta.2020.0209.
2. Yuan, Z.; Lai, S.; Liu, H. CoLLMLight: Cooperative Large Language Model Agents for Network-Wide Traffic Signal Control, 2026, [2503.11739].
3. Liu, P.; Fons, E.; Vapsi, A.; Ghassemi, M.; Vyetrenko, S.; Borrajo, D.; Potluru, V.K.; Veloso, M. TS-Agent: Understanding and Reasoning Over Raw Time Series via Iterative Insight Gathering. https://arxiv.org/abs/2510.07432, 2025.
4. Zhan, T.; Song, X.; Guan, T.; Pan, S.; Jin, M. AION: Next-Generation Tasks and Practical Harness for Time Series. https://arxiv.org/abs/2605.25045, 2026.
5. Ning, X.; Tieu, K.; Fu, D.; Wei, T.; Li, Z.; Bei, Y.; Zou, J.; Ai, M.; Liu, Z.; Li, T.W.; et al. Code as Agent Harness. https://arxiv.org/abs/2605.18747, 2026.
6. Lin, J.; Liu, S.; Pan, C.; Lin, L.; Dou, S.; Xi, Z.; Huang, X.; Yan, H.; Han, Z.; Gui, T.; et al. Agentic Harness Engineering: Observability-Driven Automatic Evolution of Coding-Agent Harnesses. https://arxiv.org/abs/2604.25850, 2026.
7. Liu, H.; Li, D.; Jiang, R.; Deng, J.; Ye, W.; Sekimoto, Y. TimeClaw: A Time-Series AI Agent with Exploratory Execution Learning. https://arxiv.org/abs/2605.10038, 2026.
8. Tang, Y.; Williams, A.R.; Ashok, A.; Zheng, V.Z.; Gupta, S.; Sun, L.; Drouin, A.; Laradji, I.H.; Marcotte, E.; Zantedeschi, V. Dr-CiK: A Testbed for Foresight-Driven Agents. https://arxiv.org/abs/2605.27904, 2026.
9. Chen, Y.; Qin, X.; Liu, C.; Wu, L.; Samia, N.I.; Ding, K. LLM Agents for Time-Series: A Survey. https://arxiv.org/abs/2608.26226, 2026.
10. Jalori, G.; Verma, P.; Arik, S.O. FLAIRR-TS: Forecasting LLM-Agents with Iterative Refinement and Retrieval for Time Series. https://arxiv.org/abs/2508.19279, 2025. Findings of EMNLP 2025; DOI: 10.18653/v1/2025.findings-emnlp.834.
11. Tao, X.; Wu, Y.; Cheng, M.; Guo, Z.; Gao, T. AnomaMind: Agentic Time Series Anomaly Detection with Tool-Augmented Reasoning. https://arxiv.org/abs/2602.13807, 2026.
12. Liang, Y.; Wen, H.; Nie, Y.; Jiang, Y.; Jin, M.; Song, D.; Pan, S.; Wen, Q. Foundation Models for Time Series Analysis: A Tutorial and Survey. In Proceedings of the Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, 2024, pp. 6555–6565. https://doi.org/10.1145/3637528.3671451.
13. Zhang, X.; Chowdhury, R.R.; Gupta, R.K.; Shang, J. Large Language Models for Time Series: A Survey. https://arxiv.org/abs/2402.01801, 2024.
14. Jiang, Y.; Pan, Z.; Zhang, X.; Garg, S.; Schneider, A.; Nevmyvaka, Y.; Song, D. Empowering Time Series Analysis with Large Language Models. https://arxiv.org/abs/2402.03182, 2024.
15. Jiang, Y.; Ning, K.; Pan, Z.; Shen, X.; Ni, J.; Yu, W.; Schneider, A.; Chen, H.; Nevmyvaka, Y.; Song, D. Multimodal Time Series Analysis: A Tutorial and Survey. KDD Tutorial and Survey, 2025. https://doi.org/10.1145/3711896.3736567.
16. Zhang, R.; Goh, S.Q.; Chen, X.; Li, Z.; Wen, H.; Guo, J.; Yang, M.; Yin, H.; Yang, Q.; Yiu, S.M.; et al. From Prompts to Agents: A Comprehensive Survey of LLM-Driven Time Series Analysis. https://ssrn.com/abstract=6614598, 2026. Companion resource repository: https://github.com/CoderPowerBeyond/Agent-Prompt-TS-Survey, https://doi.org/10.2139/ssrn.6614598.
17. Chang, C.; Shi, Y.; Cao, D.; Yang, W.; Hwang, J.; Wang, H.; Pang, J.; Wang, W.; Liu, Y.; Peng, W.C.; et al. A Survey of Reasoning and Agentic Systems in Time Series with Large Language Models. https://arxiv.org/abs/2509.11575, 2026. Transactions on Machine Learning Research; canonical OpenReview forum l3QW42g6u3.
18. Xu, X.; Tang, J.; Chen, J.; Yan, Y.; Tang, Z.; Zhou, X.; Hu, X.; Wei, W.; Wu, J.; Chen, Q.; et al. LLM-Based Agents for Forecasting and Prediction: Methods, Training, Evaluation, and Applications. https://arxiv.org/abs/2608.23058, 2026.
19. Jin, M.; Wang, S.; Ma, L.; Chu, Z.; Zhang, J.Y.; Shi, X.; Chen, P.Y.; Liang, Y.; Li, Y.F.; Pan, S.; et al. Time-LLM: Time Series Forecasting by Reprogramming Large Language Models. In Proceedings of the ICLR, 2024, [2310.01728].

20. Xie, Z.; Li, Z.; He, X.; Xu, L.; Wen, X.; Zhang, T.; Chen, J.; Shi, R.; Pei, D. ChatTS: Aligning Time Series with LLMs via Synthetic Data for Enhanced Understanding and Reasoning, 2025, [2412.03104]. https://doi.org/10.14778/3742728.3742735.
21. Recursive Self-Improvement in AI: A Survey. Preprints.org, 2026.
22. Cai, Y.; Li, X.; Goswami, M.; Wiliński, M.; Welter, G.; Dubrawski, A. TimeSeriesGym: A Scalable Benchmark for Time Series Machine Learning Engineering Agents. https://arxiv.org/abs/2505.13291, 2025.
23. Zhao, H.; Zhang, X.; Wei, J.; Xu, Y.; He, Y.; Sun, S.; You, C. TimeSeriesScientist: A General-Purpose AI Agent for Time Series Analysis. https://arxiv.org/abs/2510.01538, 2025.
24. Kang, H.; Kim, J.; Park, J.; Kang, P. Detecting Time Series Anomalies Like an Expert: A Multi-Agent LLM Framework with Specialized Analyzers. https://arxiv.org/abs/2605.05725, 2026.
25. Zhou, S.; Xuan, Y.; Ao, Y.; Wang, X.; Fan, T.; Wang, H. MERIT: Multi-Agent Collaboration for Unsupervised Time Series Representation Learning. In Proceedings of the Findings of the Association for Computational Linguistics: ACL 2025, 2025, pp. 24011–24028. https://doi.org/10.18653/v1/2025.findings-acl.1231.
26. Yu, Y.; Li, H.; Chen, Z.; Jiang, Y.; Li, Y.; Zhang, D.; Liu, R.; Suchow, J.W.; Khashanah, K. FinMem: A Performance-Enhanced LLM Trading Agent with Layered Memory and Character Design, 2023, [2311.13743].
27. Chang, C.; Wang, W.Y.; Peng, W.C.; Chen, T.F. LLM4TS: Aligning Pre-trained LLMs as Data-Efficient Time-Series Forecasters. *ACM Transactions on Intelligent Systems and Technology* **2025**, *16*, 1–20, [2308.08469]. https://doi.org/10.1145/3719207.
28. Ravuru, C.; Sakhinana, S.S.; Runkana, V. Agentic Retrieval-Augmented Generation for Time Series Analysis. https://arxiv.org/abs/2408.14484, 2024.
29. Chow, W.; Gardiner, L.; Hallgrïmsson, H.T.; Xu, M.A.; Ren, S.Y. Towards Time-Series Reasoning with LLMs, 2026, [2409.11376].
30. Zhou, Y.; Luo, Y.; Cheng, M.; Liu, Q.; Wang, J.; Wang, D.; Chen, E. Time Series Forecasting via Reasoning: A Slow-Thinking Approach with Reinforcement Fine-Tuned LLMs. In Proceedings of the Proceedings of the 35th ACM International Conference on Information and Knowledge Management, 2026. https://doi.org/10.1145/3799682.3840737.
31. Ahamed, M.A.; Parmar, M.; Goyal, P.; Li, C.L.; Cheng, Q.; Pfister, T.; Yoon, J. Reasoning-Aware Training for Time Series Forecasting, 2026, [2605.08625].
32. Cao, D.; Gee, M.; Liu, J.; Wang, H.; Yang, W.; Wang, R.; Liu, Y. Conversational Time Series Foundation Models: Towards Explainable and Effective Forecasting, 2025, [2512.16022].
33. Tao, X.; Cheng, M.; Pan, B.; Jiang, C.; Zhang, H.; Gao, T.; Liu, Y.; Liu, Q.; Chen, E. CastFSR: A Fast–Slow–Reflect Agentic Reasoning Framework for Context-Aware Time Series Forecasting. https://arxiv.org/abs/2608.03031, 2026.
34. Wu, X.; Lu, J.; Li, Z.; Qiu, X.; Hu, J.; Guo, C.; Jensen, C.S.; Yang, B. TimeART: Towards Agentic Time Series Reasoning via Tool-Augmentation. https://arxiv.org/abs/2601.13653, 2026.
35. Sun, H.; Xu, C.; Xie, H.; Li, H.; Huang, Y.; Zhang, C.; Jin, M.; Liu, X.; Wang, G.; Bian, J. ChatAD: Reasoning-Enhanced Time-Series Anomaly Detection with Multi-Turn Instruction Evolution. https://arxiv.org/abs/2601.13546, 2026.
36. Lee, Y.; Nair, R.; Zhang, Q.; Lee, K.; Khattab, O.; Finn, C. Meta-Harness: End-to-End Optimization of Model Harnesses. https://arxiv.org/abs/2603.28052, 2026.
37. Xiao, Y.; Sun, E.; Luo, D.; Wang, W. TradingAgents: Multi-Agents LLM Financial Trading Framework, 2025, [2412.20138].
38. Kong, Y.; Yao, Q.; Nie, Y.; Li, Y.; Shao, Y.; Zohren, S.; Vettoruzzo, A.; Vanschoren, J.; Jin, M.; Wen, Q. TimeSage-MT: A Multi-Turn Benchmark for Evaluating Agentic Time Series Reasoning. https://arxiv.org/abs/2606.01498, 2026.
39. Shen, C.; Yu, W.; Wu, F.; Song, D.; Tong, H.; Luo, D.; Cheng, W.; Chen, H.; Ni, J. MetaCaster: Meta-Harness-Optimized Agent for End-to-End Few-Shot Learning of Lightweight Time Series Forecasters. https://arxiv.org/abs/2608.23473, 2026.
40. Ye, W.; Yang, W.; Cao, D.; Zhang, Y.; Tang, L.; Cai, J.; Liu, Y. TS-Reasoner: Domain-Oriented Time Series Inference Agents for Reasoning and Automated Analysis. https://arxiv.org/abs/2410.04047, 2024.
41. Yu, Y.; Yao, Z.; Li, H.; Deng, Z.; Cao, Y.; Chen, Z.; Suchow, J.W.; Liu, R.; Cui, Z.; Xu, Z.; et al. FinCon: A Synthesized LLM Multi-Agent System with Conceptual Verbal Reinforcement for Enhanced Financial Decision Making, 2024, [2407.06567].

42. Li, Z.; Jin, K.; Bei, Y.; Zou, J.; Kumar, A.; Ning, X.; Zhao, Y.; Ai, M.; Jing, B.; Tong, H.; et al. Harnessing Generalist Agents for Contextualized Time Series. https://arxiv.org/abs/2606.05404, 2026.
43. Zhu, T.; Wang, H.; Wen, Y. BatteryLake: Agentic, Physics-Grounded Curation of Heterogeneous Battery Aging Data and Benchmarking. https://arxiv.org/abs/2607.09762, 2026.
44. Ji, X.; Zhang, L.; Zhang, W.; Peng, F.; Mao, Y.; Liao, X.; Zhang, K. LEMAD: LLM-Empowered Multi-Agent System for Anomaly Detection in Power Grid Services. *Electronics* **2025**, *14*, 3008. https://doi.org/10.3390/electronics14153008.
45. Devireddy, A.; Huang, S. CALM: A Framework for Continuous, Adaptive, and LLM-Mediated Anomaly Detection in Time-Series Streams. https://arxiv.org/abs/2508.21273, 2025.
46. Ruan, W.; Liang, Y. Visual Reasoning over Time Series via Multi-Agent System. https://arxiv.org/abs/2602.03026, 2026.
47. Das, S.S.S.; Goyal, P.; Parmar, M.; Peng, N.; Tirumalashetty, V.; Li, C.L.; Zhang, R.; Yoon, J.; Pfister, T. Nexus: An Agentic Framework for Time Series Forecasting. https://arxiv.org/abs/2605.14389, 2026.
48. Chi, Y.; Chamoun, E.; Ding, Z.; Vlachos, A. WorldReasoner: Evaluating Whether Language Model Agents Forecast Events with Valid Reasoning, 2026, [2606.11816].
49. Liu, W.; Pang, S.; Wang, C.; Shen, T.; He, J.; Cui, Z.; Wu, X.; Luu, A.T.; Luo, H. TIEM: Temporal Integration of Hypergraph Evidence and Skill Memory for Event-Driven Financial Forecasting. https://arxiv.org/abs/2608.13024, 2026.
50. Yao, Q.; Kong, Y.; Nie, Y.; Li, Y.; Zohren, S.; Vettoruzzo, A.; Wen, Q.; Jin, M.; Vanschoren, J. TimeSage-EV: A Live Benchmark for Agentic Time Series Analysis in Evolving Environments. https://arxiv.org/abs/2608.14270, 2026.
51. Duan, Y.; Zhang, C.; Li, J. FactorMAD: A Multi-Agent Debate Framework Based on Large Language Models for Interpretable Stock Alpha Factor Mining. In Proceedings of the Proceedings of the 6th ACM International Conference on AI in Finance. ACM, 2025, pp. 605–613. https://doi.org/10.1145/3768292.3770377.
52. Wu, S.; Li, D.; Ye, H.; Feng, W.; Lou, J.; Zhang, B.; Feng, W.; Guo, C.; Ng, S.K. TSQAgent: Rating Time Series Data Quality via Dedicated Agentic Reasoning. https://arxiv.org/abs/2606.03629, 2026.
53. Garza, A.; Rosillo, R. TimeCopilot, 2025, [2509.00616].
54. Liao, Y.; Wang, Z.; Nie, Q.; Zhang, Z. Bridging the Last Mile of Time Series Forecasting with LLM Agents. https://arxiv.org/abs/2606.02497, 2026.
55. Pan, B.; Cheng, M.; Liu, Z.; Yu, S.; Tao, X.; Wu, Y.; Liu, Q.; Lian, D.; Chen, E. CastFlow: Learning Role-Specialized Agentic Workflows for Time Series Forecasting, 2026, [2604.27840].
56. Yang, Q.; Qiu, S.; Li, B.; Shan, X.; Feng, J.; Zhou, S.; Zhou, X.; Xing, T.; Guo, J.; Dong, X.; et al. HydroAgent: Formalizing Forecaster Expertise into Skill-Orchestrated Flood Forecasting Workflows. https://arxiv.org/abs/2607.23983, 2026.
57. Tran, T.B.; Cong, D.N.; Le, D.T.; Huynh, T.T.; Kieu, T. SCENARIODIFF: A Scenario-level Guidance Framework for Multimodal Time Series Forecasting–Extended Version. https://arxiv.org/abs/2608.17164, 2026.
58. Lee, G.; Yu, W.; Shin, K.; Cheng, W.; Chen, H. TimeCAP: Learning to Contextualize, Augment, and Predict Time Series Events with Large Language Model Agents. AAAI 2025; https://arxiv.org/abs/2502.11418, 2025.
59. Alur, R.; Stadie, B.C.; Kang, D.; Chen, R.; McManus, M.; Rickert, M.; Lee, T.; Federici, M.; Zhu, R.; Fogerty, D.; et al. AIA Forecaster: Technical Report, 2025, [2511.07678].
60. Cao, Y.; Du, Y.; Chang, Y.; Lin, L.; Chen, J. From Narrative to Auditable Forecasts: A Structured Scaffold for Agentic Forecasting, 2026, [2609.05905].
61. Li, H.; Huang, Y.H.; Xu, C.; Schlegel, V.; Jiang, R.; Batista-Navarro, R.; Nenadic, G.; Bian, J. BRIDGE: Bootstrapping Text to Control Time-Series Generation via Multi-Agent Iterative Optimization and Diffusion Modeling. In Proceedings of the Proceedings of the 42nd International Conference on Machine Learning, 2025.
62. Gu, Y.; Xiong, Y.; Mace, J.; Jiang, Y.; Hu, Y.; Kasikci, B.; Cheng, P. Argos: Agentic Time-Series Anomaly Detection with Autonomous Rule Generation via Large Language Models. https://arxiv.org/abs/2501.14170, 2025.
63. Yang, T.; Liu, J.; Siu, W.; Wang, J.; Qian, Z.; Song, C.; Cheng, C.; Hu, X.; Zhao, Y. AD-AGENT: A Multi-agent Framework for End-to-end Anomaly Detection. In Proceedings of the Findings of IJCNLP-AACL, 2025. https://doi.org/10.18653/v1/2025.findings-ijcnlp.11.
64. Kolonin, A.; Glushchenko, A.; Bochkov, E.; Saxena, A. Time Series Augmented Generation for Financial Applications. https://arxiv.org/abs/2604.19633, 2026.

65. Weng, M.; Cao, D.; Yang, W.; Sharma, Y.; Liu, Y. TemporalBench: A Benchmark for Evaluating LLM-Based Agents on Contextual and Event-Informed Time Series Tasks. https://arxiv.org/abs/2602.13272, 2026.
66. Ahamed, M.A.; Parmar, M.; Goyal, P.; Song, Y.; Le, L.T.; Cheng, Q.; Li, C.L.; Palangi, H.; Yoon, J.; Pfister, T. TFRBench: A Reasoning Benchmark for Evaluating Forecasting Systems. https://arxiv.org/abs/2604.05364, 2026.
67. Bendinelli, T.; Dox, A.; Holz, C. TraceBench: Controlled Evaluation of LLM Agents for Time-Series Root-Cause Attribution. https://arxiv.org/abs/2608.27182, 2026.
68. Jing, B.; Chen, S.; Zheng, L.; Liu, B.; Li, Z.; Zou, J.; Wei, T.; Liu, Z.; Zeng, Z.; Qiu, R.; et al. TSAQA: Time Series Analysis Question and Answering Benchmark. https://arxiv.org/abs/2601.23204, 2026.
69. Yin, Y.; Xiao, Z.; Li, M.; Liu, Y.; Nan, S.; He, Y.; Wang, R.; Zhang, Z.; Liao, Q.; Gu, Y. MMTS-BENCH: A Comprehensive Benchmark for Time Series Understanding and Reasoning. https://arxiv.org/abs/2602.08588, 2026.
70. Jin, H.; Kim, K.; Kwon, J. GridMind: LLMs-Powered Agents for Power System Analysis and Operations, 2025, [2509.02494].
71. Zhang, Y.; Saber, A.M.; Youssef, A.; Kundur, D. Grid-Agent: An LLM-Powered Multi-Agent System for Power Grid Control, 2025, [2508.05702].
72. Nnamdi, M.C.; Marteau, B.L.; Zhong, Y.; Tamo, J.B.; Wang, M.D. Tree-of-Evidence: Efficient System 2 Search for Faithful Multimodal Grounding. https://arxiv.org/abs/2604.07692, 2026.
73. Lin, M.; Chen, Z.; Liu, Y.; Zhao, X.; Wu, Z.; Wang, J.; Zhang, X.; Wang, S.; Chen, H. Decoding Time Series with LLMs: A Multi-Agent Framework for Cross-Domain Annotation. https://arxiv.org/abs/2410.17462, 2024.
74. Jiang, Y.; Yu, W.; Lee, G.; Song, D.; Shin, K.; Cheng, W.; Liu, Y.; Chen, H. TimeXL: Explainable Multi-Modal Time Series Prediction with LLM-in-the-Loop. https://arxiv.org/abs/2503.01013, 2025.
75. Xu, C.; Liu, Z.; Li, Z. FinArena: A Human-Agent Collaboration Framework for Financial Market Analysis and Forecasting. https://arxiv.org/abs/2503.02692, 2025.
76. Zhang, Y.; Feng, Y.; Li, D.; Zhang, K.; Chen, J.; Deng, B. Can Competition Enhance the Proficiency of Agents Powered by Large Language Models in the Realm of News-driven Time Series Forecasting? https://arxiv.org/abs/2504.10210, 2025.
77. Zhang, J.; Feng, L.; Guo, X.; Wu, Y.; Dong, Y.; Xu, D. TimeMaster: Training Time-Series Multimodal LLMs to Reason via Reinforcement Learning. https://arxiv.org/abs/2506.13705, 2025.
78. Yang, Y.; Liu, Z.; Song, L.; Ying, K.; Wang, Z.; Bamford, T.; Vyetrenko, S.; Bian, J.; Wen, Q. Time-RA: Towards Time Series Reasoning for Anomaly Diagnosis with LLM Feedback. https://arxiv.org/abs/2507.15066, 2025.
79. Yeh, C.C.M.; Lai, V.; Saini, U.S.; Fan, X.; Fan, Y.; Wang, J.; Dai, X.; Zheng, Y. Empowering Time Series Forecasting with LLM-Agents. https://arxiv.org/abs/2508.04231, 2025.
80. Li, Z.; Chen, B.; Xue, H.; Salim, F.D. ZARA: Training-Free Motion Time-Series Reasoning via Evidence-Grounded LLM Agents. https://arxiv.org/abs/2508.04038, 2025.
81. Guan, T.; Meng, Z.; Li, D.; Wang, S.; Yang, C.H.H.; Wen, Q.; Liu, Z.; Siniscalchi, S.M.; Jin, M.; Pan, S. TimeOmni-1: Incentivizing Complex Reasoning with Time Series in Large Language Models. https://arxiv.org/abs/2509.24803, 2025. Accepted at ICLR 2026.
82. Parker, F.; Chan, N.; Zhang, C.; Ghobadi, K. Eliciting Chain-of-Thought Reasoning for Time Series Analysis using Reinforcement Learning. https://arxiv.org/abs/2510.01116, 2025.
83. Sui, S.; Xu, Z.; Hu, X. Training-Free Time Series Classification via In-Context Reasoning with LLM Agents. https://arxiv.org/abs/2510.05950, 2025.
84. Zhang, X.; Gao, T.; Cheng, M.; Pan, B.; Guo, Z.; Liu, Y.; Tao, X.; Liu, Q. AlphaCast: A Human Wisdom-LLM Intelligence Co-Reasoning Framework for Interactive Time Series Forecasting. https://arxiv.org/abs/2511.08947, 2025.
85. Sukhorukov, D.; Zakharov, A.; Glazkov, N.; Yanchanka, K.; Kirilin, V.; Dubovitsky, M.; Sultimov, R.; Maksimov, Y.; Makarov, I. A Modular LLM-Agent System for Transparent Multi-Parameter Weather Interpretation. https://arxiv.org/abs/2512.11819, 2025.
86. Chae, J.; Wang, R.; Xiong, C.; Gong, Y.; Zhang, L.; Ji, J.; Yu, D.; Qin, P. Auditable Context-Aware HFMD Forecasting with Structured LLM Agents. https://arxiv.org/abs/2511.23276, 2026. First public version posted 2025-11-28; title updated in v2.
87. Ni, J.; Wang, S.; He, Q.; Jin, M.; Jin, W. STReasoner: Empowering LLMs for Spatio-Temporal Reasoning in Time Series via Spatial-Aware Reinforcement Learning. https://arxiv.org/abs/2601.03248, 2026.

88. Trirat, P.; Kwak, J.M.; Heo, J.; Lee, H.; Hwang, S.J. Multimodal Collaborative Debate for Zero-Shot Time Series Reasoning, 2026, [2601.19151].
89. Zhou, J.; Li, D.; Li, B.; Zhang, X.; Meng, E.; Li, L.; Chen, Z.; Lou, J.; Ng, S.K. Time Series Reasoning via Process-Verifiable Thinking Data Synthesis and Scheduling for Tailored LLM Reasoning. https://arxiv.org/abs/2602.07830, 2026.
90. Tao, X.; Cheng, M.; Jiang, C.; Gao, T.; Zhang, H.; Liu, Y. Cast-R1: Learning Tool-Augmented Sequential Decision Policies for Time Series Forecasting. https://arxiv.org/abs/2602.13802, 2026.
91. Zhang, Z.; Zhang, J.; Liu, H.; Lv, Q.; Yang, J.; Cai, K.; Wang, K. AgriWorld: A World Tools Protocol Framework for Verifiable Agricultural Reasoning with Code-Executing LLM Agents. https://arxiv.org/abs/2602.15325, 2026.
92. Messica, S.; Zhang, J.; Li, K.; Tsiligkaridis, T.; Zitnik, M. Adaptive Time Series Reasoning via Segment Selection. https://arxiv.org/abs/2602.18645, 2026.
93. Lee, H.; Cho, Y.; Kwak, S.S.; Calvo, R.A. SAGE: Sensor-Augmented Grounding Engine for LLM-Powered Sleep Care Agent. https://arxiv.org/abs/2604.16342, 2026.
94. Liu, S. Optimizing Multi-Agent Weather Captioning via Text Gradient Descent: A Training-Free Approach with Consensus-Aware Gradient Fusion. https://arxiv.org/abs/2603.21673, 2026.
95. Gwiazda, M.; Cai, Y.; Goswami, M.; Choudhry, A.; Dubrawski, A. TimeSeriesExamAgent: Creating Time Series Reasoning Benchmarks at Scale. https://arxiv.org/abs/2604.10291, 2026.
96. Li, L.; Huang, J.; Quan, Q.; Li, D.; Li, B.; Zhang, X.; Meng, E.; Feng, W.; Lou, J.; Ng, S.K. Empowering VLMs for Few-Shot Multimodal Time Series Classification via Tailored Agentic Reasoning. https://arxiv.org/abs/2605.09395, 2026.
97. Feng, K.; Shan, Z.; Fang, Y.; Tan, Y.; Lu, S.; Gu, S.; Ma, L.; Lu, X.; Ren, K. KairosAgent: Agentic Time Series Forecasting with Fused Semantic Reasoning. https://arxiv.org/abs/2605.30002, 2026.
98. Wang, Z.; Liu, X.; Yang, W.; Zhou, K.; Zhang, Y.; Jia, W. ODTQA-FoRe: An Open-Domain Tabular Question Answering Dataset for Future Data Forecasting and Reasoning. https://arxiv.org/abs/2606.02433, 2026.
99. Spinelli, F.; Municio, E.; Baguer, P.; Garcia-Aviles, G.; Costa-Perez, X. DAST: A VLM-LLM Framework for Cross-Interface Anomaly Detection in O-RAN. https://arxiv.org/abs/2606.06261, 2026.
100. Lin, X.; Yin, D.; Prabowo, A.; Xue, H.; Hu, W.; Razzak, I.; Amos, M.; Behrens, S.; Salim, F.D. TopoBrick: Agentic Topology Sampling of Exogenous Variables for Zero-Shot Building IoT Forecasting. https://arxiv.org/abs/2607.06349, 2026.
101. Gopali, S.; Chhetri, B.; Giri, D.; Siami-Namini, S.; Namin, A.S. Neuro-Agentic Control: A Deep Learning-based LLM-Powered Agentic AI Framework for Controlling Security Controls. https://arxiv.org/abs/2607.09076, 2026.
102. Rashid, M.; Hong, L.; Ding, J.; Hossain, K.S.M.T. Fin-Analyst at FinMMEval 2026 Task 3: A Live Hybrid Trading Agent with LLM Specialists and Rule-Based Signals. https://arxiv.org/abs/2607.12233, 2026.
103. Keleşoğlu, N.; Sobczak, Ł.; Domańska, J. From Plots to Words: Model-Aware Multimodal Explanations as a Foundation for Accessible, Non-Visual Interaction. https://arxiv.org/abs/2608.24910, 2026.
104. Lopes, L.G.O.; de Almeida Vieira, T.M.; de Lima Junior, E.T.; Lira, W.W.M. An Explainable LLM Agent Layer for Open-World Anomaly Detection in Oil Wells. https://arxiv.org/abs/2608.04041, 2026.
105. Zumarraga, N.; Steno, L.; Wang, N.; Rosenblattl, M.; Kaar, T.; Xu, M.A.; O'Sullivan, K.; Kreft, M.; Fleisch, E.; Schmiedmayer, P.; et al. TimeRLM: Recursive Language Models Enable Precise Anomaly Localization in Long-Context Time-Series. https://arxiv.org/abs/2608.03391, 2026.
106. Kang, J.; Han, S.; Choe, H.; Lee, S. Traceable Multi-Agent System for Knowledge-Based Forecasting. https://arxiv.org/abs/2608.03339, 2026.
107. Wang, F.; Zhang, Z.; Zhuang, D.; He, L.; Wang, S.; Mo, B.; Yu, P.S. When Do Corrective Features Help? An Agent for Corrective Feature Discovery on Black-Box Forecasters. https://arxiv.org/abs/2608.05207, 2026.
108. Sarker, M.F.; Namira, A.R.; Alam, W.B.; Arefeen, M.A.; Emu, M.; Nimi, S.T. QuanTiMedAI: Quantum-Enhanced Time-Series Model Guided by Agentic AI for Cardiac Arrest Mortality Prediction. https://arxiv.org/abs/2608.06294, 2026.
109. Hasan, T.; Ghanta, M.; Sarkar, S.; Guin, U. AgenticTwin: An Agentic LLM Framework Integrated with Digital Twin for Anomaly Detection. https://arxiv.org/abs/2608.11679, 2026.
110. Guo, J.; Huang, S.; Gao, Y.; Li, Z.; Ge, J.; Liang, S.; Kuang, X.; Wang, M. AQuA: Recursively Self-Improving Quantitative Trading Research Agents. https://arxiv.org/abs/2608.12841, 2026.
111. Yang, Z.; Xu, C.; Wang, Y.; Gu, T.; Yang, H.; Lin, Y.; Liu, K.; Xiao, F. ReasonCast: Agentic Demand Forecasting with Selective Semantic Reasoning. https://arxiv.org/abs/2608.15291, 2026.

112. Jiang, L.; Wei, Y.; Xi, X.; Langham-Lopez, J.; Bao, Y.; Khraishi, R.; Ang, Y.; Tung, A.K.H.; Szpruch, L.; Ni, H. EvoTS-Agent: A Self-Evolving LLM Agent for Financial Time Series Change Point Detection. https://arxiv.org/abs/2608.17933, 2026.
113. Pesta, M.; Ting, Y.S. Agentic Active Learning Meets Visual Embeddings: Finding Anomalies among 370 000 Variable Stars from ASAS-SN. https://arxiv.org/abs/2608.23688, 2026.
114. Sukhorukov, D.; Zakharov, A.; Glazkov, N.; Yanchanka, K.; Kirilin, V.; Dubovitsky, M.; Sultimov, R.; Maksimov, Y.; Makarov, I. Hierarchical AI-Meteorologist: LLM-Agent System for Multi-Scale and Explainable Weather Forecast Reporting. https://arxiv.org/abs/2511.23387, 2025.
115. Han, L.; Yang, K.; Wang, O.; Wu, J.; Quan, P.; Dong, G.; Mulayim, O.B.; Ma, S.; Yuan, Y.; Hong, D.; et al. TS-Skill: A Benchmark for Evaluating Analytical Skills in Time-Series Question Answering. https://arxiv.org/abs/2605.24703, 2026.
116. Wang, X.; Feng, M.; Qiu, J.; Gu, J.; Zhao, J. From News to Forecast: Integrating Event Analysis in LLM-Based Time Series Forecasting with Reflection, 2024, [2409.17515].
117. Wan, S.C.; Wang, Y.Y.; Hou, T.; Chang, X.; Jan, A. Grammar of the Wave: Towards Explainable Multivariate Time Series Event Detection via Neuro-Symbolic VLM Agents, 2026, [2603.11479].
118. Chang, Y.; Du, Y.; Cao, Y.; Chen, J.; Lin, L. ForecastCompass: Guiding Agentic Forecasting with Adaptive Factor Memory, 2026, [2605.30858].
119. Ni, H.; Zhang, W.; Liu, F.; Lu, M.; Liu, H. SIREN: Towards End-to-End Extreme-Weather Early Warning with Experience-Grounded LLM Agents, 2026, [2607.24588].
120. Zhang, L.; Zhai, Y.; Jia, T.; Huang, X.; Duan, C.; Li, Y. AgentFM: Role-Aware Failure Management for Distributed Databases with LLM-Driven Multi-Agents, 2025, [2504.06614].
121. Lai, S.; Xu, Z.; Zhang, W.; Liu, H.; Xiong, H. LLMLight: Large Language Models as Traffic Signal Control Agents, 2025, [2312.16044].
122. Da, L.; Liou, K.; Chen, T.; Zhou, X.; Luo, X.; Yang, Y.; Wei, H. Open-TI: Open Traffic Intelligence with Augmented Language Model, 2024, [2401.00211].
123. Gruver, N.; Finzi, M.; Qiu, S.; Wilson, A.G. Large Language Models are Zero-Shot Time Series Forecasters. In Proceedings of the NeurIPS, 2023. https://doi.org/10.52202/075280-0861.
124. Wawer, M.; Chudziak, J.A. Integrating Traditional Technical Analysis with AI: A Multi-Agent LLM-Based Approach to Stock Market Forecasting. In Proceedings of the Proceedings of the 17th International Conference on Agents and Artificial Intelligence, 2025, [2506.16813]. https://doi.org/10.5220/0013191200003890.
125. Wang, L.; Duan, P.; He, Z.; Lyu, C.; Chen, X.; Zheng, N.; Yao, L.; Ma, Z. Agentic Large Language Models for day-to-day Route Choices. *Transportation Research Part C: Emerging Technologies* **2025**, *180*, 105307. https://doi.org/10.1016/j.trc.2025.105307.
126. Joshi, S. Using Gen AI Agents With GAE and VAE to Enhance Resilience of US Markets. *The International Journal of Computational Science, Information Technology and Control Engineering* **2025**, *12*, 23–34. https://doi.org/10.5121/ijcsitce.2025.12102.
127. Zhang, W.; Zhao, L.; Xia, H.; Sun, S.; Sun, J.; Qin, M.; Li, X.; Zhao, Y.; Zhao, Y.; Cai, X.; et al. A Multimodal Foundation Agent for Financial Trading: Tool-Augmented, Diversified, and Generalist, 2024, [2402.18485].
128. Xu, X.; Wang, H.; Liang, Y.; Zhao, Y.; Shu, K.; Yu, P.S. Can Multimodal LLMs Perform Time Series Anomaly Detection?, 2026, [2502.17812].
129. Feli, M.; Azimi, I.; Liljeberg, P.; Rahmani, A.M. An LLM-Powered Agent for Physiological Data Analysis: A Case Study on PPG-based Heart Rate Estimation, 2025, [2502.12836].
130. Ang, Y.; Bao, Y.; Jiang, L.; Tao, J.; Tung, A.K.H.; Szpruch, L.; Ni, H. Structured Agentic Workflows for Financial Time-Series Modeling with LLMs and Reflective Feedback, 2025, [2508.13915].
131. Emmanoulopoulos, D.; Olby, O.; Stillman, N.R.; Lyon, J. To Trade or Not to Trade: An Agentic Approach to Estimating Market Risk Improves Trading Decisions, 2025, [2507.08584].
132. Guo, Q.; Li, X.; Chen, J.; Guo, Z.; Li, X.; Zhang, L.; Li, L. A Dual Large Language Models Architecture with Herald Guided Prompts for Parallel Fine Grained Traffic Signal Control, 2025, [2511.00136].
133. Wei, S.; Wang, Q.; Yang, K. Virtual Traffic Police: Large Language Model-Augmented Traffic Signal Control for Unforeseen Incidents, 2026, [2601.15816].
134. Chudziak, J.A.; Wawer, M. ElliottAgents: A Natural Language-Driven Multi-Agent System for Stock Market Analysis and Prediction, 2025, [2507.03435].
135. Zou, X.; Yang, Y.; Chen, Z.; Hao, X.; Chen, Y.; Huang, C.; Liang, Y. Traffic-R1: Reinforced LLMs Bring Human-Like Reasoning to Traffic Signal Control Systems, 2025, [2508.02344].

136. Guo, Q.; Li, X.; Chen, J.; Guo, Z.; Xu, S.; Zhang, L.; Li, L. CuraLight: Debate-Guided Data Curation for LLM-Centered Traffic Signal Control, 2026, [2604.05663].
137. Papadakis, C.; Dimitriou, A.; Filandrianos, G.; Lymperaiou, M.; Thomas, K.; Stamou, G. ATLAS: Adaptive Trading with LLM AgentS Through Dynamic Prompt Optimization and Multi-Agent Coordination, 2026, [2510.15949].
138. Benhenda, M. FinRL-DeepSeek: LLM-Infused Risk-Sensitive Reinforcement Learning for Trading Agents, 2025, [2502.07393].
139. Li, C.; Kim, H.; Deng, W.; Jin, M.; Huang, W.; Lu, M.; Yuan, B. CLIMATEAGENT: Multi-Agent Orchestration for Complex Climate Data Science Workflows, 2025, [2511.20109].
140. Li, Z.; Yan, S.; Cao, J.; Zhang, M.; Wei, A.; Yoo, J.; Hong, Y. HydroAgent: Closing the Gap Between Frontier LLMs and Human Experts in Hydrologic Model Calibration via Simulator-Grounded RL, 2026, [2605.17792].
141. Li, Y.; Liu, S.; Liu, B. PandaAI: A Practical Agent CQ2 for Neuro-symbolic Data Analysis And Integrated Decision-Making in Quantitative Finance, 2026, [2606.06823].
142. Falouz, D.; Douaibia, A.; Bechar, A.; Elmir, Y.; Amira, A.; Oulefki, A. An Agentic AI Pipeline for Appliance-Level Energy Anomaly Detection and LLM-Driven Recommendations, 2026, [2606.28467].
143. Poikarayil, O.B.; Schockaert, C.; Nahhas, A.; Daase, C.; Dawodi, M.; Baktash, J.A. GenAutoML: An Agentic Framework for Dynamic Architecture Generation and Optimization in Time-Series Analysis, 2026, [2606.05860].
144. Li, B.; Zhao, C. S2S-FDD: Bridging Industrial Time Series and Natural Language for Explainable Zero-shot Fault Diagnosis, 2026, [2603.08048].
145. Tao, X.; Cheng, M.; Guo, Z.; Yu, S.; Liu, Y.; Liu, Q.; Wang, S. MemCast: Memory-Driven Time Series Forecasting with Experience-Conditioned Reasoning, 2026, [2602.03164].
146. Bao, Y.; Xi, X.; Liu, X.; Ge, W.; Jiang, L.; Zhang, K.; Khraishi, R.; Ang, Y.; Tung, A.K.H.; Szpruch, L.; et al. MOSAIC: Modular Orchestration for Structured Agentic Intelligence and Composition, 2026, [2606.00708].
147. Chen, Y.; Xie, H. TRACE: Unlocking the Potential of LLMs in Time Series Forecasting for Distributed Energy Resources. *IEEE Transactions on Artificial Intelligence* **2025**, *7*, 696–709. https://doi.org/10.1109/TAI.2025.3572853.
148. Wang, B.; Zhou, Y.; Ge, L.; Kung, S.Y. Large-model-based smart agent for time series anomaly detection in power systems. *Expert Systems with Applications* **2026**, *296*, 128917. https://doi.org/10.1016/j.eswa.2025.128917.
149. Guo, J.; Huang, S.; Li, S.; Gao, Y.; Cheng, Z.; Ge, J.; Liang, S.; Li, Z.; Lu, H.; Yin, M.; et al. Can Language Models Learn to Forecast Stock Prices, 2026, [2609.36914].
150. Goel, S.; Chandak, N.; Arun, A.; Prabhu, A.; Staab, S.; Hardt, M.; Andriushchenko, M.; Geiping, J. FutureSim: Replaying World Events to Evaluate Adaptive Agents, 2026, [2605.15188].
151. Chandak, N.; Goel, S.; Prabhu, A.; Hardt, M.; Geiping, J. Scaling Open-Ended Reasoning to Predict the Future, 2025, [2512.25070].

Appendix A Module-Level Contribution Ledger

The machine-readable JSON ledger is a central research artifact of this survey. It records, for each work, the bibliographic identity, publication date, claim, source locator and quotation, LLM-side contribution, General Harness dimensions, General Time-Series Harness dimensions, Task-Specific Harness dimensions, task family, work type, and review status. The ledger is extensible: each new paper adds a supported module-level record to the construction hierarchy. The human-readable profiles below expose the same information in a form suitable for inspection. The function view additionally links each module to the four temporal reading categories in Section 5, with explicit applicability decisions for model-side contributions, general runtime mechanisms, broad resources, and task procedures outside those categories. Function tags preserve the original scope, detailed mechanism labels, task memberships, and source evidence. Source-derived interface summaries are kept separately from proposed reuse. The cards also retain author-provided repository, project, data, and evaluation links together with their source locations and availability checks. These links connect a module description to its original development resources.

Appendix A.1 Paper Card schema

The contribution layers in Figure 3 supply the coordinates for the Paper Card below.

A Paper Card is the evidence unit used to maintain the survey. It keeps bibliographic identity, the paper's own claim, source evidence, module contributions, task contract, evaluation evidence, and review status in one record. The card is multi-label: one work may contribute to the LLM side and to several Harness modules. Aggregate Harness plots apply the deepest-layer rule to each concrete module, while the inclusive evidence fields remain available for audit.

Paper Card fields

Identity	Authors, year, landing page, BibTeX key, PDF, and source status.
Claim	The central claim and concrete contribution stated by the work.
Evidence	Page or section locator, quotation, figure or table, and verification status.
Modules	LLM-side capability, General Harness, General Time-Series Harness, and Task-Specific Harness entries.
Task contract	Forecasting, augmentation, anomaly diagnosis, decision support, or another supported task family.
Evaluation	Process validity, temporal-contract checks, task metrics, and deployment or decision evidence.

We construct each card in four passes: verify the work identity and publication record; extract the paper's claim with a page- or section-level quotation; code the concrete mechanisms it changes; and record task and evaluation evidence. A mechanism that transfers across temporal tasks is coded as General Time-Series Harness, a task-bound state, action, reward, or validator as Task-Specific Harness, and a domain-independent runtime mechanism as General Harness. The card can therefore support citation management, module tables, cumulative plots, and open-problem analysis without assigning a paper to a single mutually exclusive class.

Current ledger snapshot. The database contains **138** examined records: 133 paper-supported works, 2 repository-supported resources, and 3 background records excluded from contribution counts. The 135 retained records contain 290 source-located modules: 47 LLM-side, 56 General Harness, 40 General Time-Series Harness, and 147 Task-Specific Harness modules. Task memberships comprise 74 forecasting/prediction, 11 augmentation/synthesis, 35 anomaly/diagnosis, and 34 decision-support records; additional task values include classification, imputation, annotation, and question answering. Each module has one layer and may have several mechanism and task labels.

The coding unit is a concrete module contribution. A paper may contribute to several modules and may therefore appear in several dimensions. For Harness plotting, the exclusive layer is the deepest applicable one: a task-bound module is counted as Task-Specific, an otherwise reusable temporal module as General Time-Series, and only a domain-independent runtime module as General Harness. The inclusive descriptions are retained as source evidence, while every claim remains tied to a paper locator and an explicit review status.

Each row in Table A1 is a work-level record whose module and task fields can contain several entries. Read the claim together with the evidence status: this table illustrates the human-readable view, and the JSON ledger retains the page locators and quotations used to audit it.

Table A1. Human-readable view of representative module-level records from the JSON ledger. The complete profiles follow; source locators and quotations are retained in the machine-readable record.

Work	Claim	LLM	Harness	Task	Evidence
ChatTS [20]	Synthetic temporal instruction data and alignment for time-series understanding and reasoning.	Alignment; instruction tuning	–	Cross-task reasoning	PDF locator recorded
TimeSeries Scientist [23]	Role-based curation, planning, forecasting, validation, and reporting workflow.	–	GTS; Task-Specific	Forecasting	Section 3.2–3.3
TIMECLAW [42]	Auditable temporal tools, episodic experience, and capability evolution in a time-series runtime.	–	GH; GTS	Temporal infrastructure	Sections 3.2–3.4
FinCon [41]	Multi-agent financial evidence synthesis for portfolio decision making.	–	Task-Specific	Decision support	PDF locator recorded

The five functions in Table 3 organize the Harness chapters. In the cards, detailed control, verification, feedback, and coordination labels map to planning and orchestration. Fine-grained labels name concrete procedures within those dimensions. A reviewed module has a stable identifier, one contribution layer, one or more mechanism labels, task memberships, and source evidence. Its function view records applicable temporal functions and the supporting evidence indices; selected interfaces also summarize the source-defined inputs, outputs, and scope boundary. Dimension labels and task memberships do not create additional contribution units.

Three examined works remain background records: GridMind [70] studies static power-system optimization and contingency scenarios; Grid-Agent [71] addresses static power-flow violation repair; and Tree-of-Evidence [72] uses encoder-based evidence search without an LLM Agent. Their source evidence is retained in the review history, and they contribute zero modules to the statistics.

The complete work-level profiles are provided here. The main text uses aggregate patterns and representative examples. Repository-only resources are identified with footnotes.

[a] Salesforce AI Research, *MoiraiAgent* implementation: GitHub repository.

[b] *LLM-TSFD* author repository: GitHub repository.

Table A2 lists the retained works and their source-reviewed modules. Each row can contain modules from several layers. The source JSON additionally records each module's role, scope rationale, task memberships, and exact evidence locator; its stable module identifiers reproduce the timeline without multiplying a module by its labels.

Table A2. Profiles of the 135 retained works. Each profile lists concrete modules at their assigned layers, including the independent LLM side. SM denotes a system or method and B an evaluation benchmark; † marks works whose authors do not use the Agent label.

Work	Mechanisms
Agentic RAG for TSA [28] Aug'24; S–M Forecasting and prediction; Anomaly detection and diagnosis; imputation; time series classification	**GH:** ReAct master-agent routing. **LLM:** Task-specific PEFT and direct preference optimization; Differentiable temporal prompt-pool conditioning.
TS-Reasoner [40] Oct'24; S–M Forecasting and prediction; Anomaly detection and diagnosis; imputation; temporal causal analysis	**GH:** Execution-error plan revision. **GTS:** Unified time-series operator suite. **TSK:** Within-instance forecast selection.
TESSA [73] Oct'24; S–M time series annotation	**GTS:** Multichannel temporal feature extraction. **TSK:** Domain annotation generation and review.
Argos [62] Jan'25; S–M Anomaly detection and diagnosis	**GH:** Execution-feedback syntax repair. **TSK:** Executable anomaly-rule generation; False-positive and false-negative detector correction.
TimeCAP [58] Feb'25; S–M Forecasting and prediction	**TSK:** Multimodal event-example retrieval and prediction fusion.
TimeXL [74] Mar'25; S–M Forecasting and prediction	**TSK:** Prototype explanation-conditioned predictive fusion; Reflection-driven context refinement.
FinArena [75] Mar'25; S–M Forecasting and prediction; Decision support	**GH:** Uncertainty-triggered external retrieval. **TSK:** Risk-preference-conditioned financial report.
Competition Agents [76] Apr'25; S–M Forecasting and prediction	**TSK:** Competitive news-rule evolution. **LLM:** Agent-specific forecast LLM fine-tuning.
Time-R1 [30]† Jun'25; S–M Forecasting and prediction	**LLM:** Reinforcement fine-tuning for temporal reasoning.
TimeMaster [77]† Jun'25; S–M temporal question answering; time series classification	**LLM:** Structured time-series reasoning post-training.
Time-RA [78]† Jul'25; S–M Anomaly detection and diagnosis	**TSK:** Detection, category and explanation evaluation. **LLM:** RATS40K multimodal anomaly SFT.
DCATS [79] Aug'25; S–M Forecasting and prediction	**TSK:** Metadata-guided forecasting dataset selection.
ZARA [80] Aug'25; S–M time series classification	**TSK:** Activity-pair statistical retrieval and pruning.
FLAIRR-TS [10] Aug'25; S–M Forecasting and prediction	**TSK:** Retrieval and error-driven prompt refinement.
CALM [45] Aug'25; S–M Anomaly detection and diagnosis	**GH:** In-flight model-artifact swapping. **GTS:** Ordered event-time window construction. **TSK:** Semantic drift curation for anomaly adaptation.
TimeCopilot [53] Aug'25; S–M Forecasting and prediction	**TSK:** Diagnostic forecasting model orchestration; Unified forecasting model hub.
TimeOmni-1 [81]† Sep'25; S–M Forecasting and prediction; Decision support; causal discovery; temporal question answering	**LLM:** Temporal-prior SFT and task-grounded RL; TSR-SUITE reasoning task construction.
COUNTS [82]† Oct'25; S–M Forecasting and prediction; temporal question answering; time series classification	**LLM:** Residual-VQ temporal vocabulary; DAPO time-series reasoning training.

Work	Mechanisms
TimeSeriesScientist [23] Oct'25; S–M Forecasting and prediction	**GTS:** Temporal diagnostics and preprocessing. **TSK:** Validation-driven forecaster selection.
FETA [83] Oct'25; S–M time series classification	**TSK:** Exemplar-grounded channel classification and fusion.
TS-Agent [3] Oct'25; S–M Forecasting and prediction; Anomaly detection and diagnosis; temporal question answering	**GH:** Iterative reasoner and evidence log. **GTS:** Numeric time-series analytical operators. **TSK:** Intent-specific temporal QA evidence gate.
AlphaCast [84] Nov'25; S–M Forecasting and prediction	**TSK:** Model-linked historical-case forecasting context; Context-guided forecast adjustment.
AI-Meteorologist (companion reports) [85] Nov'25; S–M Decision support	**GH:** Code-generated report illustrations. **TSK:** Meteorological interpretation and hazard bulletin.
HFMD Structured Agents [86] Nov'25; S–M Forecasting and prediction	**TSK:** HFMD event-impact interpretation; Context-conditioned count forecasting and calibration.
STReasoner [87] Jan'26; S–M Forecasting and prediction; Augmentation and synthesis; temporal question answering	**TSK:** Scenario-constrained network-SDE synthesis. **LLM:** Graph-conditioned temporal alignment and reasoning training.
ChatAD [35] Jan'26; S–M Anomaly detection; temporal dialogue and cross-task generalization	**LLM:** Multi-agent temporal instruction construction; LoRA supervision and TKTO reasoning training.
TimeART [34] Jan'26; S–M Forecasting and prediction; Anomaly detection and diagnosis; temporal question answering	**GH:** Thought-action-observation tool loop. **GTS:** Atomic temporal analysis toolkit. **LLM:** Experience and reflection-supervised temporal reasoner.
TS-Debate [88] Jan'26; S–M Forecasting and prediction; Anomaly detection and diagnosis; imputation; temporal question answering; time series classification	**GH:** Modality-isolated evidence debate. **GTS:** Time-frequency charts and numerical lookup; Temporal-scope-aware claim verification.
MAS4TS [46] Feb'26; S–M Forecasting and prediction; Anomaly detection and diagnosis; imputation; time series classification	**GH:** Confidence-gated shared agent memory. **GTS:** Plot-anchor and latent-trajectory reasoning. **TSK:** Task-routed temporal operator chain.
TemporalBench [65] Feb'26; Bench Forecasting and prediction; temporal question answering	**GTS:** Event-boundary temporal evaluation pipeline.
VeriTime [89]† Feb'26; S–M Anomaly detection and diagnosis; temporal question answering; time series classification	**LLM:** TSRgen process-verifiable reasoning data; Difficulty-scheduled reasoning fine-tuning.
AnomaMind [11] Feb'26; S–M Anomaly detection and diagnosis	**TSK:** Coarse-to-fine tool-guided interval diagnosis. **LLM:** Anomaly detector SFT and reinforcement learning.
Cast-R1 [90] Feb'26; S–M Forecasting and prediction	**GH:** Incremental decision-state memory. **TSK:** Preparation-forecasting-refinement workflow. **LLM:** Two-stage forecasting-policy training.
AgriWorld [91] Feb'26; S–M Forecasting and prediction; Anomaly detection and diagnosis; Decision support; temporal question answering	**GH:** Execute–observe–refine code loop. **GTS:** Typed spatiotemporal data and tool artifacts. **TSK:** Simulator-grounded intervention validation. **LLM:** Agricultural Qwen LoRA adaptation.
ARTIST [92]† Feb'26; S–M temporal question answering	**TSK:** Iterative temporal-segment acquisition. **LLM:** Hierarchical segment-selection reasoning policy.
SAGE-Sleep [93] Mar'26; S–M Decision support; temporal question answering	**GTS:** Queryable temporal sensor-data layer. **TSK:** Personal-baseline sleep monitoring.
WeatherTGD [94] Mar'26; S–M time series captioning	**GH:** Consensus and unique-feedback fusion. **TSK:** Specialist weather-caption refinement.
TFRBench [66] Apr'26; Bench Forecasting and prediction	**TSK:** Forecast-reference construction and reasoning evaluation.

Work	Mechanisms
TimeSeriesExamAgent [95] Apr'26; Bench temporal question answering	**GH:** Capability-aligned template filtering. **TSK:** Temporal question-template generation.
TSAG [64] Apr'26; S–M Forecasting and prediction; Anomaly detection and diagnosis; temporal question answering	**GH:** Deterministic tool-call evaluation. **GTS:** Financial temporal-tool invocation interface.
SAGE-AD [24] May'26; S–M Augmentation and synthesis; Anomaly detection and diagnosis	**GTS:** Precise-tool and compressed-LLM temporal views. **TSK:** Anomaly-family analysis and evidence aggregation; Normal-reference anomaly injection for ICL.
MarsTSC [96] May'26; S–M time series classification	**TSK:** Class-pair knowledge-bank refinement; Two-pass pseudo-label and deferred memory updates.
TimeClaw-EEL [7] May'26; S–M Forecasting and prediction; temporal question answering; time series classification	**GH:** Explore-compare-distill execution learning; Conflict-aware structured experience and injection.
Nexus [47] May'26; S–M Forecasting and prediction	**GTS:** Value-linked chronological context. **TSK:** Macro/micro forecast synthesis; Validated backtest critique guidelines.
AION [4] May'26; Bench Forecasting and prediction; temporal analysis	**GH:** Lifecycle roles and communication protocols. **GTS:** Temporal admissibility checks; Temporal task and data interface.
Dr-CiK [8] May'26; Bench Forecasting and prediction	**TSK:** Controlled forecasting-research environment; Multi-hop supporting and distracting document construction.
KairosAgent [97] May'26; S–M Forecasting and prediction	**TSK:** Morphology-conditioned forecasting bridge. **LLM:** Tool-grounded morphology SFT and turn-level RL.
TimeSage-MT [38] May'26; Bench Forecasting and prediction; Augmentation and synthesis; Anomaly detection and diagnosis; Decision support; imputation; temporal question answering; time series classification	**GH:** Planner-executor-validator reporting runtime. **GTS:** Multi-turn temporal workflow evaluation; Temporal analytical skill library.
Last-Mile [54] Jun'26; S–M Forecasting and prediction	**TSK:** Immutable baseline and constrained forecast edits; Outcome-grounded forecast revision memory; Event-window map-reduce revision planning.
TimeFore [98] Jun'26; Bench Forecasting and prediction; temporal question answering	**GH:** Execution-feedback SQL repair. **TSK:** Imputation-then-prediction tool workflow; Future-table forecasting and reasoning benchmark.
TSQAgent / TSQBench [52] Jun'26; S–M data quality assessment	**GTS:** Dimension-wise temporal quality assessment; Controlled temporal quality comparison benchmark. **LLM:** Temporal quality-dimension policy training.
TimeClaw-Harness [42] Jun'26; S–M Forecasting and prediction; question answering	**GH:** Held-out execution gate for new tools. **GTS:** Runtime-native temporal objects and tools; Signal-text episodic retrieval.
DAST [99] Jun'26; S–M Anomaly detection and diagnosis	**GTS:** Aligned cross-interface KPI profiling. **TSK:** Domain-scored anomaly localization and impact.
BatteryLake [43] Jul'26; S–M Forecasting and prediction	**GH:** Quote-grounded metadata extraction with human review. **GTS:** Canonical temporal measurement schema and validation. **TSK:** SOH/RUL reproducible benchmark package.
TopoBrick [100] Jul'26; S–M Forecasting and prediction	**TSK:** Topology-guided exogenous variable routing and verification; Past-known versus future-known exogenous contract.
Neuro-Agentic Control [101] Jul'26; S–M Decision support	**TSK:** Bounded draining proposals and forecast perturbation.
Fin-Analyst [102] Jul'26; S–M Decision support	**GH:** Prompt-keyed specialist response cache. **TSK:** Recency-weighted specialist trading aggregation.
From Plots to Words [103] Jul'26; S–M Forecasting and prediction; Decision support	**GH:** Template-aware plan execution through tools. **TSK:** Model-aware forecast explanation.
HydroAgent [56] Jul'26; S–M Forecasting and prediction	**GH:** Explicit skill contracts and deterministic executors. **TSK:** Analog flood judgment with physical and human checks; Flood-type-indexed calibration and rolling simulation.

Work	Mechanisms
CastFSR [33] Aug'26; S–M Forecasting and prediction	**TSK:** Fast-slow-reflect forecast revision. **LLM:** Distilled SFT and multi-turn policy training.
Oil-Well Explainable Agent [104] Aug'26; S–M Anomaly detection and diagnosis	**TSK:** Sensor-metric/class-profile diagnostic layer.
TimeRLM [105] Aug'26; S–M Anomaly detection and diagnosis	**GH:** External-context recursive Python sandbox. **TSK:** Long-signal anomaly localization evaluation. **LLM:** Verifiable anomaly-reward policy training.
TraceMAS [106] Aug'26; S–M Forecasting and prediction	**GTS:** Temporal availability and source consistency review. **TSK:** Versioned CLD-to-data-to-forecast lineage.
CRAFTER [107] Aug'26; S–M Forecasting and prediction	**TSK:** Residual feature discovery and correction gate; Validation-selected post-hoc corrector.
QuanTiMedAI [108] Aug'26; S–M Forecasting and prediction	**TSK:** Outcome-guided clinical feature and severity selection.
AgenticTwin [109] Aug'26; S–M Anomaly detection and diagnosis; Decision support	**TSK:** Digital-twin residual anomaly event generation; Historical fault retrieval and grounded mitigation.
AQuA [110] Aug'26; S–M Forecasting and prediction; Decision support	**GH:** Manager-mediated specialist handoffs; Immutable evaluator and validation-only search feedback. **TSK:** Causal formulaic-alpha operator grammar.
TIEM [49] Aug'26; S–M Forecasting and prediction	**GTS:** Evidence and skill ancestry time gates. **TSK:** Outcome-informed catalyst skill retrieval.
TimeSage-EV [50] Aug'26; Bench temporal question answering; time series reporting	**GH:** Reusable Python skill induction and retrieval. **GTS:** Sequential evidence-cutoff evaluation; Next-period skill activation.
ReasonCast [111] Aug'26; S–M Forecasting and prediction	**TSK:** Skip/Basic/Tool semantic forecast intervention. **LLM:** Schema and forecast-utility reasoner training.
SCENARIODIFF [57] Aug'26; S–M Forecasting and prediction	**TSK:** Historical context, scenario and anchor guidance; Scenario-conditioned diffusion forecasting.
EvoTS-Agent [112] Aug'26; S–M Anomaly detection and diagnosis	**GH:** Validation-controlled trajectory evolution. **TSK:** Change-sensitive EDA and detector selection.
Agentic Active Learning for Variable Stars [113] Aug'26; S–M Anomaly detection and diagnosis	**TSK:** LLM labeling and neighbor-propagated anomaly ranking; Class-conditional light-curve outlier scoring.
MetaCaster [39] Aug'26; S–M Forecasting and prediction; Augmentation and synthesis	**GH:** Training-job scheduling and recovery. **TSK:** Few-shot training-series generation; Forecast-utility-driven harness optimization.
TraceBench [67] Aug'26; Bench Anomaly detection and diagnosis	**GH:** Agent exploration and resource trace metrics. **TSK:** Controlled simulator-root-cause evaluation.
MoiraiAgent[a] –; S–M Forecasting and prediction	**TSK:** Context-aware forecasting and expert selection.
Hierarchical AI-Meteorologist [114] Nov'25; S–M Forecasting and prediction; time series reporting	**GH:** Cached REST artifacts and API fallback. **GTS:** Variable-aware multiscale temporal aggregation. **TSK:** Evidence-linked weather reporting.
TimeSeriesGym [22] May'25; Bench Forecasting and prediction; temporal question answering; time series classification; Anomaly detection and diagnosis	**GH:** Task resources, execution and artifact grading. **GTS:** Time-series engineering challenge collection.
TS-Skill [115] May'26; Bench temporal question answering	**TSK:** Skill-controlled temporal QA evaluation; Signal-grounded QA generation and verification.
From News to Forecast [116] Sep'24; S–M Forecasting and prediction	**TSK:** Validation-guided news selection refinement. **LLM:** News-conditioned forecasting LoRA.
Grammar of the Wave [117] Mar'26; S–M Anomaly detection and diagnosis	**TSK:** Event-logic schema and visual grounding.
ForecastCompass [118] May'26; S–M Forecasting and prediction	**TSK:** Adaptive predictive-factor and calibration memory.

Work	Mechanisms
SIREN [119] Jul'26; S–M Forecasting and prediction; Decision support	**GH:** Guarded Python execution server; Case-retrieval instruction template; Reference-guided rehearsal and skill refinement. **TSK:** End-to-end warning-chain evaluation; Task-specific hazard-predictor development.
BRIDGE [61] Jul'25; S–M Augmentation and synthesis	**TSK:** Multi-agent text-template optimization for synthesis; Text/prototype-conditioned diffusion generation.
AD-AGENT [63] Dec'25; S–M Anomaly detection and diagnosis	**GH:** Executable code generation and dry-run repair; Shared task state and cached model documentation. **TSK:** Anomaly-library pipeline construction.
AgentFM [120] Apr'25; S–M Anomaly detection and diagnosis; Decision support	**GH:** Runtime system-role reassignment. **GTS:** Metric preprocessing and temporal descriptions. **TSK:** Detection-diagnosis-mitigation orchestration.
LLMLight [121] Dec'23; S–M Decision support	**TSK:** Traffic observation-to-signal control interface. **LLM:** Critic-filtered control imitation fine-tuning.
CoLLMLight [2] Mar'25; S–M Decision support	**TSK:** Spatiotemporal cooperative signal selection; Congestion-conditioned reasoning-depth selection. **LLM:** Simulation-derived signal-policy fine-tuning.
Open-TI [122] Jan'24; S–M Decision support	**GH:** Intent-to-tool execution interface. **TSK:** Language-policy traffic meta-control.
ChatTS [20] Dec'24; S–M temporal question answering	**LLM:** Context-aware patch-to-language alignment; Attribute-based series and TSEvol instruction generation.
Towards Time-Series Reasoning [29] Sep'24; S–M temporal question answering; time series captioning; time series classification	**LLM:** Time-series encoder and embedding projection; CoT-supervised multimodal adaptation.
LLMTime [123] Oct'23; S–M Forecasting and prediction	**LLM:** Model-aware decimal time-series representation.
Time-LLM [19] Oct'23; S–M Forecasting and prediction	**LLM:** Trainable patch reprogramming around a frozen LLM.
LLM4TS [27] Aug'23; S–M Forecasting and prediction	**LLM:** Patched numerical and multiscale temporal encoding; Autoregressive alignment and forecasting fine-tuning.
LEMAD [44] Jul'25; S–M Anomaly detection and diagnosis	**GH:** Offset-based log collection recovery. **GTS:** Timestamp and heterogeneous-stream normalization. **TSK:** Metric/log alert-to-diagnosis workflow.
ElliottAgents [124] Jun'25; S–M Forecasting and prediction; Decision support	**TSK:** Elliott-wave detection and market analysis tools; Multi-agent investment-strategy synthesis.
LLMTraveler [125] Nov'25; S–M Decision support	**TSK:** EWMATT route-experience update; Day-to-day congestion feedback simulation.
LLM-TSFD[b] Mar'25; S–M Anomaly detection and diagnosis	**GH:** Conversational DataFrame code execution.
GenAI Risk Analyst [126] Jan'25; S–M Augmentation and synthesis; Decision support	**TSK:** LLM-generated scenario-model refinement queries.
FinCon [41] Jul'24; S–M Decision support	**TSK:** Manager–analyst information aggregation; Directional portfolio allocation and CVaR alerts; PnL-grounded conceptual prompt updates.
FinAgent [127] Feb'24; S–M Decision support	**GH:** Vector-indexed summaries and query retrieval. **TSK:** Market and trade-outcome reflection.
Can Multimodal LLMs Perform TSAD [128] Feb'26; S–M Anomaly detection and diagnosis	**TSK:** Anomaly granularity and modality evaluation; Anomaly-type-aware detector routing and refinement.
PPG Physiological Agent [129] Feb'25; S–M physiological signal estimation	**GH:** Planner–executor orchestration and replanning. **TSK:** Integrated PPG quality/reconstruction/peak pipeline.
Structured Agentic Workflows [130] Aug'25; S–M Forecasting and prediction; Augmentation and synthesis	**GH:** Warm-up and round-robin code-refinement search. **GTS:** Temporal model and metric resource bank.

Work	Mechanisms
To Trade or Not to Trade [131] Jul'25; S–M Augmentation and synthesis; Decision support	**TSK:** Builder-critic stochastic simulator discovery; Simulator-risk-informed trading decisions.
HeraldLight [132] Oct'25; S–M Decision support	**TSK:** Queue and approaching-vehicle duration guidance. **LLM:** Critic-corrected score-ranking fine-tuning.
Virtual Traffic Police [133] Jan'26; S–M Decision support	**TSK:** Incident-to-controller-parameter reasoning; Verified traffic incident experience retrieval.
ElliottAgents [134] Jul'25; S–M Forecasting and prediction; Decision support	**GH:** Hierarchical asynchronous task delegation. **TSK:** Elliott-wave tool analysis and backtesting.
FinMem [26] Dec'23; S–M Decision support	**TSK:** Time-sensitive financial memory with promotion; Risk-conditioned trading action policy.
Traffic-R1 [135] Aug'25; S–M Decision support	**TSK:** Parity-scheduled intersection communication. **LLM:** Expert-informed and online traffic-policy RL.
CuraLight [136] Apr'26; S–M Decision support	**TSK:** RL-filtered traffic-action debate and data selection. **LLM:** Priority-aware traffic-controller adaptation.
TradingAgents [37] Dec'24; S–M Decision support	**GH:** Structured report-based shared communication. **TSK:** Bullish/bearish research and risk deliberation.
ATLAS [137] Oct'25; S–M Decision support	**GH:** Template-preserving prompt optimization. **TSK:** Analyst-conditioned order execution.
FinRL-DeepSeek [138] Feb'25; S–M Decision support	**TSK:** News-conditioned trading-action scaling; News-conditioned portfolio-risk return modulation.
MERIT [25] Jul'25; S–M Augmentation and synthesis	**TSK:** Retrieval, augmentation selection and view review.
CLIMATEAGENT [139] Nov'25; S–M Anomaly detection and diagnosis; scientific temporal analysis	**GH:** Shared workflow state and checkpoint serialization; Plan-to-specialist execution orchestration. **TSK:** Climate analysis workflow benchmark.
HydroAgent Calibration [140] May'26; S–M hydrologic model calibration	**TSK:** Hydrologic parameter-simulation-evaluation episode. **LLM:** Hydrologic simulator-grounded LLM post-training.
PandaAI [141] Jun'26; S–M Decision support	**TSK:** Financially constrained alpha-factor tree search; Regime-conditioned portfolio and execution control; Trading-feedback symbolic constraint induction. **LLM:** Regime soft-token alignment and finance-reasoning adaptation.
Smart Energy Agent [142] Jun'26; S–M Anomaly detection and diagnosis; Decision support	**TSK:** Reliability-aware appliance diagnosis and reporting.
CastFlow [55] Aug'26; S–M Forecasting and prediction	**GTS:** Statistical and spectral temporal profiling. **TSK:** Validated forecast-strategy memory and workflow. **LLM:** Specialized evidence-guided forecast LLM.
GenAutoML [143] Jun'26; S–M Forecasting and prediction; Anomaly detection and diagnosis	**GH:** Runtime architecture discovery and hot-swapping. **GTS:** Adopted temporal normalization wrapper. **TSK:** Forecasting shape-contract validation.
S2S-FDD [144] Mar'26; S–M Anomaly detection and diagnosis	**TSK:** Fault-directed sensor selection and temporal summary; Historical fault retrieval and iterative sensor inspection.
MemCast [145] Feb'26; S–M Forecasting and prediction	**TSK:** Outcome-derived hierarchical forecasting memory; Wisdom-scored forecast exploration and constraint repair.
MOSAIC [146] Jun'26; S–M Forecasting and prediction; Augmentation and synthesis	**GH:** Repository-grounded blueprint composition; Failure-aware code-refinement policy. **GTS:** Temporal statistical and visual task profiling.
TRACE [147] Feb'25; S–M Forecasting and prediction	**TSK:** DER-specific reasoning-step selection; Forecast-consensus refinement and factuality scoring.
FactorMAD [51] Oct'25; S–M Forecasting and prediction; Decision support	**GTS:** Past-only rolling-window constraint. **TSK:** Perspective-diverse alpha-factor debate; Alpha-factor implementation and diversity validation.
Large-model-based smart agent [148] Feb'26; S–M Anomaly detection and diagnosis	**GH:** Stored self-reflections for iterative prompt improvement. **GTS:** Differencing and sliding-window preprocessing. **TSK:** History-, statistics-, and feature-enhanced anomaly judgment.
AIA Forecaster [59] Nov'25; S–M Forecasting and prediction	**GH:** Adaptive multi-step evidence search. **TSK:** Probability reconciliation and calibration.

Work	Mechanisms
Can Language Models Learn to Forecast Stock Prices [149] Sep'26; S–M Forecasting and prediction	**TSK:** Decision-date market investigation sandbox. **LLM:** Assistant-trajectory SFT and terminal-return PPO.
Conversational TS Foundation Models [32] Dec'25; S–M Forecasting and prediction	**TSK:** Forecasting ensemble optimization and continuation control; Weight-optimization audit and reflective explanation. **LLM:** SFT and GRPO adaptation of the ensemble judge.
AUDITFORECAST [60] Sep'26; S–M Forecasting and prediction	**TSK:** Base-probability and factor-odds forecasting scaffold.
FutureSim [150]† May'26; Bench Forecasting and prediction	**TSK:** Chronological event-forecast revision environment.
STRIDE [31]† May'26; S–M Forecasting and prediction	**LLM:** Temporal reasoning distillation into the language model; Joint language-numerical latent fusion.
OpenForecaster [151] Dec'25; S–M Forecasting and prediction	**TSK:** Resolution-date-bounded static news retrieval. **LLM:** Open-ended forecast policy training.
WorldReasoner [48] Jun'26; Bench Forecasting and prediction	**GTS:** Simulated-date retrieval gateway. **TSK:** Outcome, source, and hindsight-event evaluation.